Programming I *with Python*

程序设计基础 I（Python）

The content of this handbook was developed and polished by ZHANG Feng together with multiple LLM models and AI tools.

本课程手册由 ZHANG Feng 借助多种大语言模型及 AI 工具编撰打磨而成。

For full Python learning, refer primarily to the official website: https://docs.python.org/

若要完整学习 Python，请优先参考 Python 官方文档：https://docs.python.org/

2026 · First Edition

Programming I
with Python

程序设计基础 I（Python）

Period	Task Description	Contributors
2025.02-07	Provide teaching framework, demonstration demos and lecture draft notes	Z.F.
2025.02-07	Detailed refinement & Generate Markdown-format slides	ChatGPT, Gemini, Kimi
2025.02-2026.03	Build a command-line AI assistant (designed for the final project of Programming 1)	Z.F., Kimi
2026.10	Build and refine this handbook	Workbuddy, Gemini, Z.F.

Workflow for using Workbuddy:

1. First, prepare slides, practice guidebook, and course introduction.
2. Next, Gemini generates an initial draft of the handbook for each weekly chapter, with both English and Chinese versions produced directly.
3. Prepare six files in Markdown format: preview, introduction (Syllabus), slides, chapter-wise draft handbook, practice materials, and final project demo.
4. Finally, an application called "Workbuddy" (AI agent tool) compiles and polishes them into the finished handbook (textbook). (around 50 min)
5. Iterative manual revision of this handbook.

2026.11
Z.F.

Programming I with Python

程序设计基础 I（Python）

First Edition, 2026

© Prepared for teaching use at Hainan Bielefeld University of Applied Sciences.

Course website: www.bioinfo-lab.com/courses/

Code examples target Python 3.7 and later.

Preface

前言

This book exists because of a question that first-year students ask every year: *why does an introductory programming course not start with writing Python?* Why Linux, terminals, and remote servers? Why a plain text editor like nano instead of a polished graphical IDE? Why so much on functions, lists and dictionaries, and not a single word about classes and object-oriented programming?

这本书的存在，源于一年级学生每年都会问的一个问题：*为什么编程入门课不先教写 Python？*为什么要学 Linux、终端和远程服务器？为什么用 nano 这样朴素的文本编辑器，而不用漂亮的图形化 IDE？为什么讲了那么多函数、列表、字典，却只字未提类和面向对象？

These are fair questions, and this preface is the long answer. Programming is not typing code into an editor. It is knowing **where your code runs**, **where your files live**, and **what an error message actually means**. Most online tutorials open with "install an IDE, click Run, print Hello World." That path feels fast, but it hides the operating system, the file system, and the whole chain between your keystrokes and the running program. When something breaks — and something always breaks — an IDE-only learner has nowhere to stand.

这些都是合理的问题，而这篇前言就是完整的回答。编程并不是"往编辑器里打代码"，而是清楚地知道：**代码运行在哪里、文件存放在哪里、报错信息到底在说什么**。网上大多数教程的第一页是"安装 IDE，点运行，打印 Hello World"。这条路看起来很快，却把操作系统、文件系统、以及从键盘到程序运行的整条链路都藏了起来。一旦出错——而出错是必然的——只会用 IDE 的学习者将无处立足。

So this book takes the slower road on purpose. It begins on a remote Linux server, in a terminal, with a handful of small commands. It then builds Python upward from that ground: variables, control flow, functions, data structures, files, exceptions, and libraries. Each layer rests on the one below it. Object-oriented programming is deliberately left for the next course, because a class is only meaningful once you have felt the friction of organizing data with functions and dictionaries — friction you will experience in the projects here.

所以本书有意选择了那条更慢的路。它从一台远程 Linux 服务器开始，从终端里的几条小命令开始，然后在地基之上逐层搭建 Python：变量、控制流、函数、数据结构、文件、异常、库。每一层都踩在下一层之上。面向对象被刻意留给下一门课——因为只有当你真正体会过"用函数和字典组织数据"的摩擦之后，类才有意义，而这本书的项目会让你亲身体会这种摩擦。

The book is bilingual by design, not by decoration. Every idea is stated in English first, then restated in Chinese. The English carries the terminology you will meet in documentation, error messages and international teams; the Chinese makes sure that language never becomes the obstacle between you and the concept. Read both. When they disagree in your head, that is exactly where understanding begins.

本书的中英双语是设计，而不是装饰。每个概念先用英文陈述，再用中文重述。英文承载着你将来在文档、报错信息和国际化团队中会遇到的专业术语；中文则保证语言永远不会成为你和概念之间的障碍。两种都读。当它们在你脑中不一致时，那正是理解开始的地方。

One habit this book asks of you above all others: **type everything yourself**. Run every command, write every script, break it on purpose, and read the traceback with your own eyes. Programming is a craft, and no amount of watching replaces the feeling of your own hands on the keyboard. The weekly report you will submit is not bureaucracy — it is the evidence of that practice, and years from now it will still tell you what you could do.

本书对你最重要的一个要求是：**所有内容都亲手敲一遍**。运行每一条命令，写每一个脚本，故意弄坏它，亲眼读一遍报错信息。编程是一门手艺，看多少遍都替代不了指尖的体感。你要提交的周报不是形式主义——它是练习的凭证，多年以后它仍能告诉你自己曾经会做什么。

Welcome to the terminal. Let us begin.

欢迎来到终端。我们开始吧。

ZHANG Feng,

October 2026

How to Use This Book

本书使用说明

Structure

全书结构

The book mirrors the fifteen weeks of the course and is organized into six parts: foundations (Linux, first programs, data types, operators, strings), control flow, abstraction (functions and modules), data structures, the real world (files, encoding, exceptions, libraries, concurrency), and the final project. Seventeen chapters are followed by five appendices: a command reference, the Python keywords, a field guide to error messages, the course map and grading rules, and a bilingual glossary.

本书对应课程的十五个教学周，共分六个部分：基础（Linux、第一个程序、数据类型、运算符、字符串）、控制流、抽象（函数与模块）、数据结构、与真实世界对接（文件、编码、异常、第三方库、并发），以及期末项目。十七章之后是五个附录：命令速查、Python 保留关键字、错误信息速查、课程安排与考核规则，以及中英术语表。

Conventions

排版约定

- **Learning objectives** open every chapter and state what you should be able to *do* afterwards.

  每章开头的**学习目标**说明学完本章后你应当*能够做到*什么。

- **Think About It** boxes contain questions without standard answers; they train design judgement, not memorization.

  想一想栏目收录没有标准答案的问题，训练的是设计判断力，而非记忆。

- **Caution** boxes mark the traps that beginners actually fall into.

  注意栏目标注的是初学者真实会踩的坑。

- Shaded blocks are code you can type in exactly as printed. Commands for the Linux terminal and code for Python files are both shown this way.

底纹段落中的代码可以原样输入。Linux 终端命令与 Python 脚本代码均以此方式呈现。

- **Exercises** close each chapter; the recommended ones for your weekly report are listed in Appendix (Course Map and Assessment).

  每章以练习收尾；周报推荐题目见附录（课程安排与考核）。

Tools

工具

You will need three things: the Termius app (free version) to reach the course Linux server, your student account on that server, and a text editor — which, for this course, is `nano` on the server itself. Everything else is provided. Outside of phase tests, you are encouraged to use AI assistants freely: ask them to explain errors, review your code and quiz you. During phase tests, all AI tools are off.

你需要准备三样东西：用来连接课程 Linux 服务器的 Termius（免费版即可）、你在服务器上的学生账号，以及一个文本编辑器——在本课程中，它就是服务器上的 `nano`。其余一切均已备齐。除阶段测验外，鼓励自由使用 AI 助手：让它解释报错、检查代码、向你提问。阶段测验期间，所有 AI 工具一律关闭。

Table of Contents / 目录

Foundations / 基础 *9*

Linux and the Command Line / Linux 与命令行基础 10

Your First Python Programs / 第一个 Python 程序 19

Variables and Data Types / 变量与数据类型 26

Operators and Expressions / 运算符与表达式 32

Working with Strings / 字符串操作 38

Control Flow / 控制流 *45*

Making Decisions / 条件判断 46

Going in Loops / 循环结构 52

Abstraction / 抽象 *59*

Functions / 函数 60

Modules and the Standard Library / 模块与标准库 68

Data Structures / 数据结构 *75*

Lists and Tuples / 列表与元组 76

Dictionaries and Sets / 字典与集合 83

The Real World / 与真实世界对接 *91*

Reading and Writing Files / 文件读写 92

File Systems and Encodings / 文件系统与字符编码 98

Errors and Exception Handling / 错误与异常处理 104

Third-Party Libraries and pip / 第三方库与 pip 113

Doing Several Things at Once / 多任务处理 118

The Final Project / 期末项目 *123*

The Python Movie Scriptwriter / Python 电影编剧程序 124

Reference / 附录与参考资料 *130*

Linux Command Quick Reference / Linux 命令速查 131

Python Reserved Keywords / Python 保留关键字 133

A Field Guide to Error Messages / 错误信息速查 134

Course Map and Assessment / 课程安排与考核 136

Glossary / 术语表 139

PART I

Foundations / 基础

Part I builds the ground everything else stands on. Chapter 1 teaches you to reach a remote Linux server and to move around its file system with confidence. Chapters 2 through 5 then introduce the Python language itself: how programs are written and run, how data is stored in variables, how operators turn data into results, and how text — the most common data of all — is taken apart and put back together.

第一部分搭建其余一切所依赖的地基。第 1 章教你连接远程 Linux 服务器，并自信地在它的文件系统中移动。第 2 章到第 5 章随后正式引入 Python 语言本身：程序如何编写和运行，数据如何存进变量，运算符如何把数据变成结果，以及文本——最常见的数据——如何被拆解与重组。

Linux and the Command Line / Linux 与命令行基础

> *The command line is not a relic. It is the one interface that every server in the world still speaks.*
>
> 命令行不是遗物。它是这个世界上每一台服务器仍在使用的通用语言。

LEARNING OBJECTIVES 学习目标

- Explain the difference between GUI and CLI, and when each is the better tool.
  解释 GUI 与 CLI 的区别，以及各自更适合的场景。
- Connect to a remote Linux server with Termius using IP, port, username and password.
  使用 IP、端口、用户名和密码，通过 Termius 连接远程 Linux 服务器。
- Describe what SSH and SFTP are for.
  说明 SSH 与 SFTP 的用途。
- Use `pwd`, `ls`, `cd`, `mkdir`, `touch`, `cat`, `less`, `cp`, `mv` and `rm` to navigate and manage files.
  使用 `pwd`、`ls`、`cd`、`mkdir`、`touch`、`cat`、`less`、`cp`、`mv`、`rm` 完成文件导航与管理。
- Edit a text file with nano and save it.
  使用 nano 编辑并保存文本文件。
- Use Tab completion and `history` to work faster.
  使用 Tab 补全与 `history` 提升效率。

1.1 GUI, CLI and the Terminal

图形界面、命令行与终端

Before we dive into Linux, it is worth asking how a human being actually talks to a computer. There are exactly two answers, and both are still in daily use.

在深入 Linux 之前，值得先问一个问题：人究竟是怎样和计算机对话的？答案只有两个，而且两个至今都在日常使用。

- **GUI (Graphical User Interface)**: you interact with visual elements — windows, icons, buttons — using a mouse. This is what you use every day on Windows or macOS.

GUI（图形用户界面）：用鼠标与窗口、图标、按钮等视觉元素交互。这是你每天在 Windows 或 macOS 上使用的界面。

- **CLI (Command Line Interface)**: you interact with the computer exclusively by typing text commands.

  CLI（命令行界面）：完全通过输入文本命令与计算机交互。

A **terminal** is a text-based tool that lets you control a computer or server by typing commands instead of clicking icons. It is the most familiar example of a CLI. In professional software development and server management, the terminal is not optional — it is the everyday workplace.

终端（Terminal）是一种基于文本的工具，让你通过输入命令而不是点击图标来控制计算机或服务器。它是 CLI 最常见的代表。在专业软件开发与服务器管理中，终端不是可选项，而是日常工作的现场。

> **THINK ABOUT IT** 想一想
>
> What are the advantages and disadvantages of GUI and CLI? Write down your intuitive guess before reading on.
>
> 凭直觉猜一猜：GUI 和 CLI 各自的优缺点是什么？先写下答案，再往下读。
>
> GUI is easy to learn, highly visual, and forgiving — but slow and tedious for repetitive or complex work. CLI is extremely fast, powerful and easily automated, but it has a steep learning curve. Professionals end up needing both, and servers almost always offer only the second.
>
> GUI 易学、直观、容错性好，但面对重复性或复杂工作时又慢又繁琐。CLI 极快、强大、易于自动化，但学习曲线陡峭。专业人士两者都需要，而服务器几乎只提供后者。

1.2 Connecting to a Remote Server

连接远程服务器

Most of the time, Linux does not run on your laptop. It runs on remote servers — machines in machine rooms that you never touch physically. To control them, we use a terminal application and a network protocol.

大多数时候，Linux 并不运行在你的笔记本电脑上，而是运行在远程服务器上——那些你永远不会亲手触碰的机房里的机器。要控制它们，我们需要一个终端应用和一个网络协议。

Termius is a popular, cross-platform terminal application. It is the tool this course uses to connect to and manage the Linux server. Download the free version from termius.com or the app store — the free tier is fully sufficient; do not subscribe.

Termius 是一款流行的跨平台终端应用，也是本课程用来连接和管理 Linux 服务器的工具。可从 termius.com 或应用商店下载免费版——免费版完全够用，请勿付费订阅。

To create a new host connection in Termius (or in any SSH client), you need exactly four elements:

在 Termius（或任何 SSH 客户端）中创建主机连接，需要且只需要四个要素：

Element	Meaning
IP address IP 地址	The network address of the server. 服务器的网络地址。
Port 端口	The specific channel for the connection; SSH usually uses 22. 连接使用的通道；SSH 默认通常是 22。
Username 用户名	Your account name on the server. 你在服务器上的账户名。
Password 密码	Your secret key for authentication. 用于身份验证的密钥。

Two protocols do the work behind the scenes:

在背后完成这项工作的是两个协议：

- **SSH (Secure Shell)** is an encrypted network protocol used to securely access, manage, and transfer files between computers over unsecured networks. It ensures that your commands and data cannot be easily intercepted.

  SSH（安全外壳协议）是一种加密的网络协议，用于在不安全的网络上安全地访问、管理计算机并在计算机之间传输文件，确保你的命令和数据不会被轻易截获。

- **SFTP (SSH File Transfer Protocol)** is a secure file transfer protocol that runs over SSH. Termius offers a built-in SFTP panel for visual file management — uploading and downloading with drag and drop.

  SFTP（SSH 文件传输协议）是运行在 SSH 之上的安全文件传输协议。Termius 内置了 SFTP 面板，可以拖拽上传下载，进行可视化的文件管理。

1.3 The File System and Paths

文件系统与路径

> **Q: What are the two software components closest to users in a computer?**

> **问：计算机中离用户最近的两个软件组件是什么？**

The **file system**, where your data lives, and the **application system**, the programs you run. Almost everything you do on a computer is one of these two things: handling data or running programs.

一个是存放数据的**文件系统**，一个是运行程序的**应用系统**。你在计算机上做的几乎所有事，都属于这两者之一：处理数据，或运行程序。

> **Q: What are the two most important things when controlling a computer?**

> **问：控制计算机时，最重要的两件事是什么？**

Position — "where you are" in the file system — and **intention** — "what you want to do". Every command you will ever type answers these two questions.

位置——"你在哪里"，以及**意图**——"你想做什么"。你今后输入的每一条命令，都在回答这两个问题。

Absolute and relative paths 绝对路径与相对路径

To define your position, or to locate a file, we use paths.

为了定义位置或定位文件，我们使用路径。

An **absolute path** is the full path from the root directory to a file or folder. It works no matter where you currently are.

绝对路径是从根目录开始直到目标文件或文件夹的完整路径。无论你当前身在何处，它都有效。

```
Linux:   /home/zhangfeng/download/test.txt
Windows: D:\download\test.txt
```

A **relative path** is the path from your *current working directory* to the target. It starts wherever you are standing.

相对路径是从你*当前工作目录*出发到达目标的路径。它从你此刻站立的位置开始。

```
Linux:   ../download/test.txt     (.. means one directory up / .. 表示上一级目录)
Windows: ..\test.txt
```

> **THINK ABOUT IT** 想一想
>
> Why do we need relative paths at all, when absolute paths always work?
>
> 绝对路径总是有效，为什么还需要相对路径?
>
> Two reasons. They are shorter and faster to type. And they are **portable**: if you move or copy a whole folder, the relative links inside it still work. Relative paths make teamwork, project folders and navigation possible without depending on a fixed root location.
>
> 两个原因。其一，更短、输入更快。其二，具有**可移植性**：移动或复制整个文件夹后，其内部的相对引用仍然有效。相对路径让协作、项目文件夹和导航成为可能，而不依赖固定的根目录。

1.4 Essential Commands

核心命令

Now let us translate intentions into actual Linux commands. Learn the following twenty commands and you can operate any Linux server on Earth.

现在，让我们把"意图"翻译成真正的 Linux 命令。掌握以下二十条命令，你就能操作地球上任何一台 Linux 服务器。

Navigating and viewing 导航与查看

Command	Full meaning	What it does

`pwd`	print working directory 打印工作目录	Tells you your exact absolute path. 告诉你当前的绝对路径。
`ls`	list 列出	Lists files and folders here. 列出当前目录下的文件与文件夹。
`cd <path>`	change directory 切换目录	Moves you to a new location. 移动到新位置。
`cd ..`	change to parent 回到上级	Moves up one level. 向上一级。
`cd ~`	change to home 回到主目录	Returns to your user's home directory. 回到用户主目录。

```
cd ~          # go to your home directory / 回到主目录
pwd           # confirm where you are / 确认当前位置
cd ..         # step up / 向上一级
pwd
cd ~
ls            # what is around me? / 周围有什么？
```

Creating and viewing files 创建与查看文件

Command	What it does
`mkdir name`	make directory — creates a new folder. 创建一个新文件夹。
`touch f.txt`	creates a new, empty file named `f.txt`. 创建一个名为 `f.txt` 的空文件。
`cat f.txt`	concatenate and print — prints the *entire* file at once. 一次性打印文件*全部*内容。
`less f.txt`	views a file page by page; press `q` to quit. Best for large files. 分页查看文件；按 `q` 退出。适合大文件。

Copying, moving and deleting 复制、移动与删除

Command	What it does
`cp a b`	copies file `a` to `b`.

	把文件 `a` 复制为 `b`。
`cp -r fold1 fold2`	copies a folder and all its contents **r**ecursively. **递归地**复制文件夹及其全部内容。
`mv a b`	moves (or renames!) a file or folder. 移动（或重命名！）文件或文件夹。
`rm f.txt`	removes a file. There is no recycle bin. 删除文件。没有回收站。
`rm -r fold`	removes a folder and everything inside it. Use with caution! 删除文件夹及其全部内容。务必谨慎！

```
cd ~
mkdir new_folder                  # create a folder / 建一个文件夹
touch new_file.txt                # create an empty file / 建一个空文件
ls
cp /home/students/dt2026/resource/demo.txt ~/demo.txt
mv demo.txt demo_mv.txt           # rename by moving / 用移动来重命名
rm demo_mv.txt
cp -r new_folder copy_folder
rm -r copy_folder                 # careful! / 小心!
```

> **CAUTION 注意**
>
> `rm` does not move anything to a recycle bin. `rm -r` on the wrong folder is permanent, silent data loss. Before every `rm -r`, run `ls` and look at the name twice.
>
> `rm` 不会把任何东西放进回收站。在错误的文件夹上执行 `rm -r` 意味着永久的、无声的数据丢失。每次执行 `rm -r` 之前，先运行 `ls`，把名字看两遍。

1.5 Text Editing with nano

用 nano 编辑文本

While GUI systems have Notepad or Word, Linux terminals have text-based editors. **nano** is the friendliest of them, and it is already on your server.

图形界面有记事本和 Word，终端里则有自己的文本编辑器。**nano** 是其中对新手最友好的一个，而且你的服务器上已经装好了。

```
nano hello.txt      # open (or create) a file / 打开（或创建）文件
```

- Type whatever you like. The bottom two lines of the screen are the menu.
  随意输入内容。屏幕最下面两行是菜单。
- **Save**: press `Ctrl + O` (Write Out), then press `Enter` to confirm the file name.
  保存：按 `Ctrl + O`，再按 `Enter` 确认文件名。
- **Exit**: press `Ctrl + X`.
  退出：按 `Ctrl + X`。

```
cat hello.txt       # check what you saved / 检查保存的内容
```

1.6 Terminal Superpowers

终端进阶技巧

Two small habits will make you visibly faster within a week.

两个小习惯能让你在一周内明显变快。

Tab completion　Tab 自动补全

Type the first few letters of a file name or command and press `Tab`; Linux completes the rest. If several options exist, press `Tab` twice to list them all. Tab is also the best defense against typos in long paths.

输入文件名或命令的前几个字母再按 `Tab`，Linux 会自动补全剩余部分。若有多个候选，连按两次 `Tab` 会全部列出。Tab 也是防止长路径打错字的最好防线。

Command history　命令历史

Type `history` to see every command you have run, numbered. Press the **up** and **down** arrow keys to walk through your previous commands and re-run them with small edits.

输入 `history` 可以看到你运行过的所有命令及编号。按上、下方向键可以翻看并微调重跑之前的命令。

> **KEY POINTS**　本章要点
>
> - GUI is visual and easy; CLI is fast, powerful and automatable. Servers speak CLI.
>   GUI 直观易用；CLI 快速、强大、可自动化。服务器说的是 CLI。

- SSH gives you an encrypted terminal to a remote server; SFTP gives you file transfer over it. Termius provides both.

  SSH 提供到远程服务器的加密终端，SFTP 在其上提供文件传输。Termius 两者兼备。

- A connection needs IP, port, username and password.

  建立连接需要 IP、端口、用户名和密码。

- Position and intention: `pwd`/`ls`/`cd` answer "where am I"; the rest of the commands do "what I want".

  位置与意图：`pwd`、`ls`、`cd` 回答"我在哪"，其余命令完成"我想做什么"。

- `rm` and `rm -r` are permanent. Look twice.

  `rm` 与 `rm -r` 不可撤销，删前看两遍。

- Tab completion and `history` are everyday accelerators.

  Tab 补全与 `history` 是每天的加速器。

EXERCISES 练习

- Install Termius, register a free account and log in to the course server. Take a screenshot of your home directory.

  安装 Termius，注册免费账号并登录课程服务器，截图你的主目录。

- Run the full sequence in Section 1.4: create a folder and a file, copy, move, rename, and delete them. Document every step.

  完成第 1.4 节的完整流程：建目录、建文件、复制、移动、重命名、删除，并记录每一步。

- Use `less` and `cat` on `/home/students/dt2026/resource/demo.txt` and explain the difference in your report.

  分别用 `less` 和 `cat` 查看 `/home/students/dt2026/resource/demo.txt`，并在报告中解释两者差别。

- Practice nano: create `notes.txt`, write three lines, save, exit, then `cat` it.

  练习 nano：创建 `notes.txt`，写三行字，保存、退出，再用 `cat` 查看。

- Upload a file to the server with Termius SFTP, then download it back.

  用 Termius 的 SFTP 上传一个文件到服务器，再下载回来。

Your First Python Programs / 第一个 Python 程序

> *A program is a conversation with a very fast, very literal listener.*
>
> 程序是一场与一位极快、又极度较真的倾听者的对话。

LEARNING OBJECTIVES 学习目标

- Run Python in interactive mode and in script mode, and know when to use each.
  以交互模式和脚本模式运行 Python，并知道各自何时适用。
- Apply the 4-space indentation rule and write useful comments.
  应用 4 空格缩进规则并撰写有用的注释。
- Name variables and functions legally, avoiding the 35 reserved keywords.
  合法命名变量与函数，避开 35 个保留关键字。
- Write and run your first script on a Linux server.
  在 Linux 服务器上编写并运行第一个脚本。
- Read a traceback and use four basic debugging techniques.
  读懂 traceback，并使用四种基本调试技巧。

2.1 Two Ways to Run Python

运行 Python 的两种方式

Python code can be executed in two primary ways depending on your needs: interactive mode and script mode. Choosing between them is your first decision as a programmer.

根据需求，Python 代码有两种基本执行方式：交互模式与脚本模式。在两者之间做选择，是你作为程序员的第一个决定。

Interactive mode 交互模式

Interactive mode is a conversation with Python. You type a line, Python answers immediately.

交互模式就像和 Python 对话：你输入一行，它立即回应。

```
$ python3
>>> print("Hello World!")
Hello World!
>>> exit()
```

- Immediate feedback; good for quick tests.

  即时反馈，适合快速试验。

- Nothing is saved — close the window and the conversation is gone.

  什么都不会保存——关掉窗口，对话就消失了。

Script mode 脚本模式

For anything larger, you write all your code into a file (a *script*) and tell Python to execute the whole file at once.

稍大一点的程序，就应该把代码全部写进一个文件（*脚本*），然后让 Python 一次性执行整个文件。

```
$ nano hello.py     # create the file with a text editor / 用文本编辑器创建文件
$ python3 hello.py  # run it / 运行
```

- Code is saved permanently; better for complex programs.

  代码被永久保存，更适合复杂程序。

- This is how every serious project works.

  一切正经项目都是这样工作的。

2.2 Core Syntax Rules

核心语法规则

Indentation: the 4-space principle 缩进：4 空格原则

Unlike languages that group code with curly braces `{}`, Python uses indentation — spaces at the start of a line. The standard is exactly **4 spaces**.

与用大括号 `{}` 组织代码的语言不同，Python 使用缩进——行首的空格——来组织代码。标准做法是严格的 **4** 个空格。

```
# Correct / 正确
if True:
    print("This is properly indented")
```

```
# Error / 错误
if True:
print("This will cause IndentationError")
```

THINK ABOUT IT 想一想

Why does Python force the 4-space principle on everyone?

为什么 Python 要强迫所有人遵守 4 空格原则?

Because indentation is not decoration — it *is* the structure of the program. Forcing every developer to indent cleanly produces a visually obvious hierarchy: you can see at a glance which lines belong together. Braces can be placed sloppily; four spaces cannot. Python trades a little freedom for a lot of readability.

因为缩进不是装饰——它*就是*程序的结构。强制整洁缩进让层级关系一目了然：哪些行属于同一块，一眼可见。大括号可以随便摆，四个空格却糊弄不了人。Python 用一点自由换来了大量可读性。

Comments 注释

Comments are notes left in the code for humans; the computer ignores them completely.

注释是写给人看的笔记，计算机会完全忽略它们。

```
# A single-line comment / 单行注释

'''
A multi-line comment
spanning several lines
多行注释
'''

def calculate():
    """A docstring: the comment attached to a function / 函数的文档字符串"""
```

THINK ABOUT IT 想一想

Why do we need comments when the code already says what it does?

代码本身已经说明了"做什么"，为什么还要注释?

Comments explain the *why* behind the code. Six months from now, "your future self" is effectively a stranger who will have to re-derive your logic; a good comment saves them — and your teammates — from deciphering every line.

> 注释解释的是代码背后的*为什么*。六个月后，"未来的你"对这段代码而言就是个陌生人，需要重新推导逻辑；一条好注释能把未来的你和你的队友从逐行破译中拯救出来。

Statement separation　语句分隔

```
x = 1; y = 2; print(x + y)   # valid but not recommended / 合法但不推荐

x = 1                        # preferred: one statement per line / 推荐：一行一句
y = 2
print(x + y)
```

2.3 Reserved Keywords

保留关键字

Python has 35 reserved keywords (as of Python 3.7). These words define the language itself and **cannot be used as variable or function names**.

Python 有 35 个保留关键字（截至 Python 3.7）。这些单词构成了语言本身，**不能用作变量名或函数名**。

Keyword				
False	None	True	and	as
assert	async	await	break	class
continue	def	del	elif	else
except	finally	for	from	global
if	import	in	is	lambda
nonlocal	not	or	pass	raise
return	try	while	with	yield

2.4 Hello World, on a Real Server

在真实服务器上运行 Hello World

Here is the complete ritual, from login to output. Every week of this course begins with these five steps.

下面是从登录到输出的完整仪式。本课程的每一周，都从这五步开始。

```
1. Log in to the Linux server with Termius.   用 Termius 登录 Linux 服务器
2. Create the file:     nano hello.py         创建文件
3. Write the code:      print('Hello World!')  编写代码
4. Save and exit:       Ctrl+O, Enter, Ctrl+X  保存并退出
5. Run it:              python3 hello.py      运行

Hello World!
```

Congratulations — you have just completed the full chain from keyboard to running program on a remote machine. Everything else in this book is a variation on this chain.

恭喜——你刚刚完成了从键盘到远程机器上运行程序的完整链路。本书其余内容，都是这条链路的变奏。

2.5 Common Errors and Debugging

常见错误与调试

When programming, you will inevitably make mistakes. Learning to read errors is a superpower — it converts frustration into diagnosis.

编程时你必然会犯错。学会读报错是一种超能力——它把挫败感变成诊断。

The three errors you will meet today　今天就会遇到的三种错误

1. **NameError** — using a name that does not exist.

   NameError（命名错误）——使用不存在的名字。

```
prin("Missing t")
# NameError: name 'prin' is not defined
```

2. **IndentationError** — breaking the spacing rules.

   IndentationError（缩进错误）——破坏了空格规则。

3. **TypeError** — mixing incompatible types.

   TypeError（类型错误）——混用了不兼容的类型。

```
"5" + 3
# TypeError: can only concatenate str (not "int") to str
```

Reading a traceback　读懂 traceback

When Python crashes, it prints a **traceback** — the roadmap of the crash.

程序崩溃时，Python 会打印 **traceback**（回溯）——一张通往事故现场的路线图。

```
Traceback (most recent call last):
  File "demo.py", line 3, in <module>
    print(undefined_var)
NameError: name 'undefined_var' is not defined
```

Part	Meaning
`Traceback`	The start of the error roadmap. 错误路线图的起点。
`File "demo.py"`	Which file the error happened in. 错误发生在哪个文件。
`line 3`	The exact line number of the crash. 崩溃的确切行号。
`NameError`	The type of error. 错误的类型。
`undefined_var`	The specific name causing the problem. 导致问题的具体名字。

Four debugging techniques 四种调试技巧

- Read the error message carefully, bottom-up: the last line names the error, the lines above locate it.
  自下而上仔细阅读报错：最后一行给出错误类型，上面的行给出位置。
- Check the named line number — and the line *above* it, since errors often surface one line late.
  检查报错行号——以及它*上面*一行，因为错误常常晚一行才暴露。
- Use `print()` statements to track the value of variables while the program runs.
  在程序运行过程中用 `print()` 追踪变量的值。
- Paste the error into an AI assistant and ask it to explain — outside phase tests this is not cheating, it is how professionals work.
  把报错贴给 AI 助手请它解释——在非测验时间这不是作弊，这正是专业工作方式。

KEY POINTS 本章要点

- Interactive mode is a scratchpad; script mode is how real programs live in files.
  交互模式是草稿纸；脚本模式才是真实程序的生存方式。

- Indentation is structure: 4 spaces, never tabs mixed with spaces.

  缩进就是结构：4 个空格，绝不与 Tab 混用。

- Comments record the *why*; the 35 keywords belong to the language, not to your variables.

  注释记录"为什么"；35 个关键字属于语言，不属于你的变量。

- Hello World on a server = login, nano, save, run.

  服务器上的 Hello World = 登录、nano、保存、运行。

- A traceback is a map: read it bottom-up, check the line, print your variables.

  traceback 是地图：自下而上读，查行号，打印变量。

EXERCISES 练习

- Enter and exit interactive mode; print the Python version with `import sys; print(sys.version)`.

  进入并退出交互模式；用 `import sys; print(sys.version)` 打印 Python 版本。

- Create `hello.py` on the server that prints two lines: a greeting and your name.

  在服务器上创建 `hello.py`，打印两行：一句问候和你的名字。

- On purpose: remove the indentation under an `if`, run it, and copy the traceback into your report with a one-sentence explanation.

  故意去掉 `if` 下面的缩进并运行，把 traceback 抄进报告并用一句话解释。

- Write `structure.py` with a function `greet(name)` returning a greeting, and call it for yourself.

  编写 `structure.py`，含函数 `greet(name)` 返回问候语，并为你自己调用一次。

- Pick one error message from this chapter and ask an AI assistant to explain it; compare its explanation with the table above.

  从本章挑一条报错信息，请 AI 助手解释，并把它的解释与上表对比。

Variables and Data Types / 变量与数据类型

> *A variable is not a box that holds a thing. It is a label you tie to a thing.*
>
> 变量不是装东西的盒子，而是你拴在东西上的一张标签。

LEARNING OBJECTIVES 学习目标

- Explain binding, garbage collection and reassignment.
  解释绑定、垃圾回收与重新赋值。
- Apply the naming rules and PEP8 conventions.
  应用命名规则与 PEP8 规范。
- Explain dynamic typing and contrast it with static typing.
  解释动态类型，并与静态类型对比。
- Distinguish the four core data types and avoid the floating-point trap.
  区分四种核心数据类型，并避开浮点数陷阱。
- Convert types explicitly and predict Python's automatic conversions.
  进行显式类型转换，并预判 Python 的自动转换。

3.1 The Nature of Variables

变量的本质

Think of variables as labeled boxes in a warehouse.

把变量想象成仓库里贴着标签的盒子。

```
box_label = "item_count"   # the variable name: the label / 变量名：标签
box_content = 17           # the stored value: the content / 存储的值：内容
```

Three mechanisms make Python variables different from the boxes of other languages:

三个机制让 Python 的变量与其他语言的"盒子"不同：

- **Binding**: a variable is a reference (a pointer) to an object in memory, not the object itself.

绑定：变量是对内存中对象的引用（指针），而不是对象本身。

- **Garbage collection**: when no variable points to some data any more, Python cleans it up automatically.

  垃圾回收：当没有任何变量指向某块数据时，Python 会自动清理它。

- **Reassignment**: rebinding a name never changes the old object — it just ties the label to a new one.

  重新赋值：重新绑定名字并不会改变旧对象，只是把标签拴到了新对象上。

```
x = 10     # create object 10, bind x to it / 创建对象 10，绑定到 x
x = "cat"  # create object "cat", rebind x / 创建新对象 "cat"，重新绑定 x
```

3.2 Variable Naming Rules

变量命名规则

A name may contain letters, digits and underscores, but **cannot start with a digit**.

名字可以包含字母、数字和下划线，但**不能以数字开头**。

```
# Valid but discouraged / 合法但不推荐（不符合 PEP8）
_privateVar = 1
MAX_LIMIT = 100
tempVar2 = 3.14

# Invalid / 非法
3attempt = 5             # SyntaxError: cannot start with a number / 不能以数字开头
special-chars = "text"   # SyntaxError: hyphen reads as minus / 连字符会被当作减号
```

PEP8 (Python Enhancement Proposal 8) is the community style guide. Its naming rule for variables is simple: use `snake_case` — lowercase words joined by underscores.

PEP8（Python 增强提案 8）是社区的编码风格指南。它对变量命名的规则很简单：使用 `snake_case`——全小写单词用下划线连接。

```
studentname -> student_name
calcVal     -> calculate_value
```

> **PRO PRACTICES** 专业习惯
>
> The interpreter does not care how you name things; your teammates do. A consistent naming style is the cheapest documentation you will ever write.
>
> 解释器并不在意你怎么命名，但队友在意。统一的命名风格是你能写出的最便宜的文档。

3.3 Dynamic Typing

动态类型

Python figures out the data type automatically when the code runs — and the same name may hold different types over its lifetime.

Python 在运行时自动推断数据类型——而且同一个名字在生命周期里可以先后装不同类型的数据。

```
dynamic_var = 100          # int (整数)
print(type(dynamic_var))   # <class 'int'>

dynamic_var = "Python"     # str (字符串)
print(type(dynamic_var))   # <class 'str'>
```

In C++ you must declare a type when the variable is born, and it is locked forever. In Python, variables change type freely.

在 C++ 中，变量出生时必须声明类型，且终身锁定。在 Python 中，变量可以自由更换类型。

> **THINK ABOUT IT** 想一想
>
> "Insufficient computing power" versus "computing power redundancy" — what does this have to do with types?
>
> "算力不足"与"算力冗余"——这和类型系统有什么关系?
>
> Today, computing-power bottlenecks mostly occur on GPUs rather than CPUs. It is true that Python's dynamic typing consumes more CPU resources than C++. Nevertheless, modern CPUs are relatively abundant for most ordinary-script workloads. Still, static-typing brings extra benefits beyond performance: compile-time error-checking and better maintainability for large projects. Dynamic-type overhead remains a real concern for CPU-intensive tasks and embedded systems.
>
> 当下算力瓶颈大多出现在 GPU（大模型、图像处理），而非 CPU。Python 的动态类型相比 C++ 确实会带来额外 CPU 开销，不过现代 CPU 算力相对充足，多数常规脚本场景能够承受该代价。但静态类型的优势不只有性能，还可在编译阶段发现类型错误，利于大型项目维护；同时 CPU 资源依然宝贵，CPU 密集计算、嵌入式场景下动态类型的开销仍是现实问题。

3.4 The Four Core Data Types

四种核心数据类型

Python's four primitive types are **integer**, **float**, **string** and **boolean**.

Python 的四种原始类型是整型、浮点型、字符串和布尔型。

Integers in other bases 其他进制的整数

```
binary_num = 0b1010     # 10  (binary / 二进制)
octal_num  = 0o177      # 127 (octal / 八进制)
hex_num    = 0x1af      # 431 (hexadecimal / 十六进制)
number     = 33         # 33  (decimal / 十进制)
```

Floating-point precision 浮点数精度

Computers store numbers in binary, and many decimals simply cannot be represented perfectly in binary — so tiny errors appear.

计算机用二进制存数，而许多小数在二进制中根本无法完美表示——于是微小的误差出现了。

```
print(0.1 + 0.2 == 0.3)         # False  !
print(f"{0.1 + 0.2:.17f}")      # 0.30000000000000004

# The fix: the Decimal library / 解决办法：Decimal 库
from decimal import Decimal
print(Decimal('0.1') + Decimal('0.2'))   # exactly 0.3 / 精确的 0.3
```

This is the single most surprising result for beginners, and it is not a Python bug — it is how every binary computer works. Money, scores and other values that must be exact should use `Decimal` or integers in the smallest unit (cents).

这是最让初学者震惊的结果，而且它不是 Python 的 bug——所有二进制计算机都是如此。

金额、成绩等必须精确的数值，应使用 `Decimal`，或用最小单位（如"分"）的整数来存储。

Strings 字符串

```
s1 = 'Single quotes'
s2 = "Double quotes"
s3 = '''Triple quotes preserve
line breaks and "both" quote types'''
```

Booleans 布尔型

```
A = True
B = False
print(A and B)   # False
print(A or B)    # True
```

3.5 Type Conversions

类型转换

Explicit conversion 显式转换

```
age = int("25")        # str -> int
pi = float("3.1415")   # str -> float
price = str(9.99)      # float -> str
```

Every value can also be asked whether it behaves as True or False (**truthiness**):

每个值都可以被追问它在布尔意义上是真是假（**真值**）：

```
bool(0)      # False — zero is false / 0 为假
bool("")     # False — empty string is false / 空字符串为假
bool([])     # False — empty list is false / 空列表为假
bool("a")    # True — any content is true / 有内容即为真
```

Automatic conversion 自动转换

In mixed arithmetic, Python promotes quietly:

在混合运算中，Python 会悄悄地做提升：

```
3 + 5.0      # int promoted to float -> 8.0 / 整型提升为浮点型
True + 10    # bool treated as int(1) -> 11 / True 被当作数字 1
```

Automatic promotion is convenient but hides information. When a `str` meets an `int`, Python refuses to guess — you must convert explicitly with `str()` or `int()`.

自动提升很方便，但也会隐藏信息。当 `str` 遇到 `int`，Python 拒绝猜测——你必须用 `str()` 或 `int()` 显式转换。

> **KEY POINTS** 本章要点
>
> - Variables are labels bound to objects; Python garbage-collects what no label points to.
>
> 变量是拴在对象上的标签；没有标签指向的对象会被 Python 回收。
>
> - Names: letters, digits, underscores; no leading digit; prefer `snake_case` (PEP8).

命名：字母、数字、下划线；不能以数字开头；推荐 `snake_case`（PEP8）。

- Dynamic typing trades machine power for human time.

  动态类型用机器算力换取人的时间。

- `0.1 + 0.2 != 0.3` is binary reality; use `Decimal` when exactness matters.

  `0.1 + 0.2 != 0.3` 是二进制的现实；需要精确时用 `Decimal`。

- Truthiness: `0`, `""`, `[]`, `None` are false; almost everything else is true.

  真值：`0`、`""`、`[]`、`None` 为假，其余几乎都为真。

- `str + int` is a `TypeError`: convert explicitly.

  `str + int` 会报 `TypeError`：请显式转换。

EXERCISES 练习

- Write `variables.py` that stores your age, height and name, prints each value together with `type()`, then converts the age to a string and prints its type again.

  编写 `variables.py`，存储你的年龄、身高、姓名，逐一打印值和 `type()`，再把年龄转成字符串打印其类型。

- Predict, then verify: `bool(0)`, `bool("0")`, `bool(-1)`, `bool([])`, `bool("False")`.

  先预测再验证：`bool(0)`、`bool("0")`、`bool(-1)`、`bool([])`、`bool("False")`。

- Explain in one sentence why `0.1 + 0.2 == 0.3` is `False`, then fix it with `Decimal`.

  用一句话解释 `0.1 + 0.2 == 0.3` 为何是 `False`，再用 `Decimal` 修复它。

- Convert `0b1010`, `0o177`, `0x1af` to decimal by hand, then check with Python.

  手工把 `0b1010`、`0o177`、`0x1af` 换算成十进制，再用 Python 验证。

Operators and Expressions / 运算符与表达式

> *An expression is a small machine: feed it values, it returns a value.*
>
> 表达式是一台小机器：喂给它值，它还给你值。

LEARNING OBJECTIVES 学习目标

- Use all seven arithmetic operators, including floor division and modulus.
  使用全部七种算术运算符，包括整除与取模。
- Compare numbers and strings correctly, including the ASCII rules.
  正确比较数字与字符串，包括 ASCII 规则。
- Combine conditions with `and`, `or`, `not` and explain short-circuiting.
  用 `and`、`or`、`not` 组合条件，并解释短路计算。
- Evaluate mixed expressions by hand using the precedence table.
  依据优先级表手工推演混合表达式。
- Use compound assignment operators and avoid the shared-object pitfall.
  使用复合赋值运算符，并避开共享对象陷阱。

4.1 Arithmetic Operators

算术运算符

Operator	Description	Example	Result
`+`	Addition 加	`5 + 3`	`8`
`-`	Subtraction 减	`7 - 2`	`5`
`*`	Multiplication 乘	`3 * 4`	`12`
`/`	True division 真除法	`10 / 3`	`3.333...`

`//`	Floor division 整除（向下取整）	`10 // 3`	`3`
`%`	Modulus 取模（求余数）	`10 % 3`	`1`
`**`	Exponentiation 幂运算	`2 ** 3`	`8`

`/` always returns a `float`, even when the division is exact: `8 / 2` is `4.0`. `//` throws the remainder away. `%` keeps only the remainder — it is the tool for "is this number even?" (`n % 2 == 0`) and for cycling through positions.

`/` 永远返回 `float`，即使整除也不例外：`8 / 2` 的结果是 `4.0`。`//` 把余数丢掉。`%` 只保留余数——它是判断"这个数是不是偶数？"（`n % 2 == 0`）以及在位置间循环取值的工具。

4.2 Comparison Operators

比较运算符

Numbers compare the way you expect: `a > b`, `a < b`, `a == b` (equal), `a != b` (not equal).

数字的比较与直觉一致：`a > b`、`a < b`、`a == b`（等于）、`a != b`（不等于）。

Strings follow three rules:

字符串则遵循三条规则：

1. Comparison proceeds character by character, using ASCII values.
   逐字符比较，依据 ASCII 码。
2. If one string is a prefix of the other, the shorter one is smaller.
   若一个是另一个的前缀，短的更小。
3. Comparison is case-sensitive.
   比较区分大小写。

```
print('Apple' < 'Banana')  # True  (A=65 < B=66)
print('cat1'  > 'cat')     # True  (longer beats its prefix / 长的前缀胜出)
print('cat'   > 'CAT')     # True  (lowercase a=97 > uppercase A=65)
print('10'    < '2')       # True  ('1'=49 < '2'=50 — strings, not numbers!)
```

That last line is the trap: `'10' < '2'` is `True` because these are strings, and the *character* `'1'` happens to sort before the character `'2'`. Convert to numbers before comparing quantities stored as text.

最后一行就是陷阱：`'10' < '2'` 为 `True`，因为它们是字符串，而*字符* `'1'` 恰好排在字符 `'2'` 前面。比较以文本存储的数量之前，先转换成数字。

4.3 Logical Operators

逻辑运算符

`and`, `or` and `not` combine boolean values. The full truth table:

`and`、`or`、`not` 组合布尔值。完整真值表如下：

X	Y	X and Y	X or Y	not X
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Short-circuiting 短路计算

Python stops evaluating a logical expression as soon as the outcome is decided. If the left side of `or` is `True`, the right side is never even looked at; if the left side of `and` is `False`, same story.

一旦结果已定，Python 立即停止计算逻辑表达式。若 `or` 的左边为 `True`，右边根本不会被看一眼；`and` 左边为 `False` 时同理。

```
def check():
    print("Called!")
    return False

print(True or check())    # True is enough; check() never runs / 短路，无输出
print(False and check())  # False ruins it; check() never runs / 短路，无输出
```

Short-circuiting is not a curiosity — it is a tool: `if user is not None and user.is_active:` never crashes, because the second test only runs when the first one passed.

短路不是冷知识，而是工具：`if user is not None and user.is_active:` 永远不会崩溃，因为只有第一项通过时第二项才会执行。

4.4 Assignment Operators

赋值运算符

Compound operators 复合运算符

```
counter = 5
counter += 3    # equivalent to: counter = counter + 3 / 等价写法
counter **= 2   # 8 ** 2 = 64
```

The chained-assignment pitfall 链式赋值陷阱

```
list1 = list2 = []   # both names point to the SAME list / 两个名字指向同一个列表
list1.append(5)
print(list2)         # [5] — surprise! / 牵一发而动全身
```

Assignment never copies. `list1 = list2 = []` creates one empty list and sticks two labels on it. If you want two independent lists, write two independent creations.

赋值从不复制。`list1 = list2 = []` 创建了一个空列表，只是贴了两张标签。想要两个独立的列表，就老老实实创建两次。

4.5 Precedence

运算符优先级

Parentheses `()` are not operators, but they always win. Inside them, the order is:

括号 `()` 不是运算符，却永远优先。括号内部的顺序是：

Priority	Operators	Note
1	`**`	Exponentiation 幂
2	`+x`, `-x`	Unary sign 正负号
3	`*`, `/`, `//`, `%`	Multiplicative family 乘除取余

4	`+`, `-`	Additive 加减
5	`<`, `>`, `==`, `!=`	Comparisons 比较
6	`not`	
7	`and`	
8	`or`	

When in doubt, add parentheses — for the reader, not for Python. `(a and b) or c` costs nothing and prevents a whole class of bugs.

拿不准时，加括号——为读者而加，不是为 Python。`(a and b) or c` 不花一分钱，却能消灭一整类 bug。

Worked example 推演实例

```
result = 5 + 3 * 2 ** 2 // (4 % 3) - 1

1. Parentheses:      (4 % 3) = 1       括号
2. Exponent:         2 ** 2  = 4       幂
3. Mult. left->right: 3 * 4 = 12; 12 // 1 = 12   乘除从左到右
4. Additive:         5 + 12 - 1 = 16   加减

result = 16
```

KEY POINTS 本章要点

- `/` gives a float, `//` floors, `%` keeps the remainder.

  `/` 给浮点，`//` 向下取整，`%` 留余数。

- String comparison is character-by-character ASCII, case-sensitive, and `'' < '2'`.

  字符串比较是逐字符 ASCII、区分大小写，且 `'10' < '2'`。

- Short-circuiting: `and`/`or` stop as soon as the answer is known.

  短路：`and`/`or` 在答案已定时立即停止。

- `a = b = []` shares one object between two names.

  `a = b = []` 让两个名字共享同一个对象。

- Precedence: `**`, then unary sign, then `* / // %`, then `+ -`, then comparisons, then `not`, `and`, `or`.

  优先级：`**`、一元正负、`* / // %`、`+ -`、比较、`not`、`and`、`or`。

EXERCISES 练习

- Write `operators.py` with `a = 10, b = 3`, printing the result of all seven arithmetic operators, then the comparisons `a > b` and `a == b`, then `x and y`, `x or y`, `not x` for `x = True, y = False`.

  编写 `operators.py`：令 `a = 10, b = 3`，打印七种算术运算结果，再打印 `a > b`、`a == b`，以及 `x = True, y = False` 时的 `x and y`、`x or y`、`not x`。

- By hand, evaluate `2 + 3 * 4 ** 2 // 5 - 1`; then verify with Python.

  手工计算 `2 + 3 * 4 ** 2 // 5 - 1`，再用 Python 验证。

- Explain why `'10' < '2'` is `True`, and write one line that compares them as numbers instead.

  解释为何 `'10' < '2'` 是 `True`，并写一行代码把它们按数字比较。

- Demonstrate the chained-assignment pitfall with two lists, then fix it.

  用两个列表演示链式赋值陷阱，然后修复它。

Working with Strings / 字符串操作

> *Most of the world's data is text pretending to be something else.*
>
> 世界上大多数数据，都是伪装成别的东西的文本。

LEARNING OBJECTIVES 学习目标

- Index and slice strings, including negative indices and steps.
  对字符串做索引与切片，包括负索引和步长。
- Use the core string methods: `split`, `join`, `strip`, `find`, `lower`, `upper`.
  使用核心字符串方法：`split`、`join`、`strip`、`find`、`lower`、`upper`。
- Choose among `+` concatenation, `format()` and f-strings.
  在 `+` 拼接、`format()` 与 f-string 之间做选择。
- Handle escape characters and raw strings.
  处理转义字符与原始字符串。

5.1 Indexing and Slicing

索引与切片

A string is a sequence of characters. Each character has a position — an **index**.

字符串是字符的序列。每个字符都有位置——即索引。

- **Positive index**: `0` for the first character, up to `len(s) - 1`.
  正向索引：第一个字符是 `0`，最后一个到 `len(s) - 1`。
- **Negative index**: `-1` is the last character, `-len(s)` the first.
  负向索引：`-1` 是最后一个字符，`-len(s)` 是第一个。

```
text = "programming"
print(text[0])      # 'p'
print(text[-1])     # 'g'  (last character / 最后一个字符)
print(len(text))    # 11
```

Slicing uses `string[start:end:step]`, and the end index is **not included**.

切片语法为 `string[start:end:step]`，且不包含结束索引。

```
text = "programming"
print(text[3:7])    # 'gram'
print(text[:3])     # 'pro'  (from the beginning / 从头开始)
print(text[7:])     # 'ming' (till the end / 到结尾)
print(text[::2])    # 'pormig' (every 2nd character / 每隔一个取一个)
print(text[::-1])   # 'gnimmargorp' — the classic reverse trick / 经典的反转技巧
```

`text[

-1] `deserves a second look: an omitted start and end mean "the whole string", and a step of -1` walks it backwards. Reversing a string is a one-liner — remember this pattern, it appears in exercises everywhere.

`text[

-1] 值得再看一眼：省略起点和终点表示"整个字符串"，步长 `-1` 表示倒着走。反转字符串只需一行——记住这个模式，各种练习里都会遇到。

5.2 The String Method Library

字符串方法库

Strings come with a large toolbox of methods. These six do most of the daily work.

字符串自带一个大工具箱。下面六个方法完成了日常的大部分工作。

Split and join　分割与拼接

```
csv = "apple,banana,cherry"
items = csv.split(",")      # ['apple', 'banana', 'cherry']
new_str = "-".join(items)   # "apple-banana-cherry"
```

`split` turns a string into a list of pieces; `join` glues a list of strings back together. They are mirror images, and together they are the backbone of text processing.

`split` 把字符串拆成列表；`join` 把字符串列表重新粘起来。二者互为镜像，共同构成文本处理的骨架。

Stripping whitespace　去除空白符

```
s = "   hello world  \t\n"      # \n newline, \t tab / \n 换行，\t 制表符
print(s.strip())    # "hello world"     (both sides / 两侧都去)
print(s.lstrip())   # left side only / 只去左侧
print(s.rstrip())   # right side only / 只去右侧
```

`strip()` is the first thing you should do to any line read from a file — invisible trailing newlines cause an amazing number of bugs.

`strip()` 是处理从文件读来的任何一行的第一步——看不见的行尾换行符制造了数量惊人的 bug。

Finding and case 查找与大小写

```
s = "Python Programming"
print(s.find('thon'))          # 2 (index of first match; -1 if absent)
print("Hello".lower())         # 'hello'
print("world".upper())         # 'WORLD'
print("title case".title())    # 'Title Case'
```

> **THINK ABOUT IT** 想一想
>
> Where do string operations matter in the real world?
>
> 字符串操作在现实世界中有哪些用武之地?
>
> **Data cleaning** (removing stray spaces, normalizing capitalization), **log parsing** (extracting error codes and timestamps from server logs), **web scraping** (locating specific text on a page) and **NLP** (breaking sentences into words for AI models). Almost every data pipeline begins with a `split` and a `strip`.
>
> 数据清洗（去多余空格、统一大小写）、日志解析（从服务器日志中提取错误码和时间戳）、网页抓取（在页面中定位特定文本）与自然语言处理（把句子切分成词供 AI 模型使用）。几乎每条数据流水线都从一次 `split` 和一次 `strip` 开始。

5.3 Formatting Strings
字符串格式化

Modern Python relies on formatting to insert variables into text. There are three generations of tools.

现代 Python 依靠格式化把变量插入文本。工具已有三代。

`+` concatenation 加号拼接

```
name = 'John'
number = 5
print("Hello, " + name + "! You have " + str(number) + " messages.")
```

Beginner-friendly and explicit, but noisy: every number needs `str()`, and one missed `+` breaks the program. This is the style this book uses in the early chapters for clarity.

对新手友好且直白，但很啰嗦：每个数字都要 `str()`，漏一个 `+` 程序就断。本书前几章为了清晰使用这种风格。

`%`-formatting and `str.format()` 旧式与 format

```
print("Hello, %s! You have %d messages." % (name, number))
print("{} + {} = {}".format(2, 3, 2 + 3))
print("{1} appears before {0}".format("apple", "banana"))  # ordering / 顺序可控
```

You will meet both in older code. Recognize them; you do not need to write them.

在旧代码里会频繁遇到这两种写法。能认出来即可，不必新写。

f-strings f-string（Python 3.6+）

```
price = 19.99
print(f"Total: ${price * 1.1:.2f}")   # Total: $21.99  (.2f = 2 decimals / 两位小数)
print(f"Binary: {42:b}")              # '101010'
```

Put an `f` before the quote and expressions go directly inside `{}` — including calculations and formatting rules. This is the modern standard; learn it after `+`, and prefer it afterwards.

在引号前加 `f`，表达式就能直接写进 `{}`——包括计算和格式规则。这是现代标准；学会 `+` 之后掌握它，此后优先使用。

Method	Readability	Performance	Notes
`+`	Low 低	Medium 中	Must convert manually 需手动转换类型
`%` / `format()`	Medium 中	Medium 中	Legacy code 常见于旧代码
f-string	High 高	Best 最佳	Expressions inline 支持内联表达式

5.4 Escape Characters and Raw Strings

转义字符与原始字符串

Some characters cannot be typed directly. The backslash `\` escapes them:

有些字符无法直接输入，反斜杠 `\` 用来转义它们：

Escape	Meaning
`\n`	Newline 换行
`\t`	Tab 制表符
`\\`	A literal backslash 一个真正的反斜杠
`\"`	A double quote inside a double-quoted string 双引号字符串里的双引号

```
print("Line 1\nLine 2")       # two lines / 两行
print("Column 1\tColumn 2")   # aligned columns / 对齐的列
print("He said \"Hello\"")    # quotes inside quotes / 引号中的引号
```

A **raw string** — an `r` before the quote — ignores escapes entirely. It is the perfect tool for Windows paths, where backslashes are literal separators:

在引号前加 `r` 得到**原始字符串**，它完全忽略转义。Windows 路径里反斜杠是真实字符，正合适：

```
path_raw = r"C:\new_folder\test.txt"
print(path_raw)     # C:\new_folder\test.txt  (no newline from \n / \n 不再换行)
```

5.5 Multi-line Strings

多行字符串

Triple quotes allow a string to span several lines without any `\n`:

三引号让字符串跨越多行而无需任何 `\n`：

```
long_text = """First line
Second line
```

```
Third line"""
print(long_text)
```

Triple-quoted strings double as multi-line comments and as docstrings inside functions — one syntax, three uses.

三引号字符串还兼任多行注释与函数文档字符串——一种语法，三种用途。

KEY POINTS 本章要点

- Index from `0` (or `-1` from the end); slices exclude the end; `[::-1]` reverses.
- `split` / `join` are mirror tools; `strip` every line you read from a file.

  `split` 与 `join` 互为镜像；从文件读来的行先 `strip`。
- Prefer f-strings; recognize `%` and `format()` in legacy code; `+` needs `str()`.

  优先 f-string；认得旧式写法；`+` 需要配 `str()`。
- `\n`, `\t`, `\\`, `\"` are escapes; `r"..."` turns them off.

  `\n`、`\t`、`\\`、`\"` 是转义；`r"..."` 关闭转义。
- Triple quotes give multi-line strings, comments and docstrings.

  三引号提供多行字符串、多行注释与文档字符串。

EXERCISES 练习

- Write `strings.py`: build `greeting = "Hello, Python!"`, print the first and last characters, the slices `[0:5]` and `[7:13]`, the stripped/upper/lower/title versions of `"  Hello World  "`, and `"Python is easy to learn".split()`.

  编写 `strings.py`：令 `greeting = "Hello, Python!"`，打印首末字符、切片 `[0:5]` 与 `[7:13]`、`"  Hello World  "` 的 strip/upper/lower/title 版本，以及 `"Python is easy to learn".split()`。
- Join `["Python", "is", "fun"]` back into one sentence, both with `join` and with `+`.

  用 `join` 和 `+` 两种方式把 `["Python", "is", "fun"]` 重新拼成一句话。
- Reverse your own name with slicing, then print its length and its uppercase form.

  用切片反转自己的名字，再打印长度和大写形式。

- Convert the + version of a greeting to an f-string and confirm both print identically.

  把 + 版问候语改写成 f-string，确认两者输出一致。

- Given `"apple,banana,cherry"`, split it, replace the commas with `" | "`, and join it back.

  给定 `"apple,banana,cherry"`，先拆分，再把逗号替换为 `" | "` 并重新拼接。

PART II

Control Flow / 控制流

A program without control flow is a straight line: same steps, same order, every time. Part II gives programs their intelligence — the ability to choose a path based on conditions, and to repeat work without repeating code. Chapters 6 and 7 cover the two great tools of this part: decisions and loops.

没有控制流的程序是一条直线：同样的步骤、同样的顺序、每次如此。第二部分赋予程序"智能"——根据条件选择路径的能力，以及不重复写代码就能重复做事的能力。第 6、7 两章覆盖本部分的两大工具：判断与循环。

Making Decisions / 条件判断

> *Every `if` is a fork in the road; your condition decides which road exists.*
>
> 每个 `if` 都是路上的岔口；你的条件决定了哪条路存在。

LEARNING OBJECTIVES 学习目标

- Write `if`, `if-else`, `if-elif-else` chains with correct colons and indentation.
  正确书写 `if`、`if-else`、`if-elif-else`，冒号与缩进无误。
- Combine conditions with comparison and logical operators.
  用比较与逻辑运算符组合条件。
- Control the order of an `elif` chain and predict which branch fires.
  控制 `elif` 链的顺序，并预判哪个分支被执行。
- Use truthiness to test empty values.
  利用真值判断空值。

6.1 The `if` Statement

`if` 语句

Think of `if` as the program's question: "what if this is true?" The syntax has three fixed parts: the keyword `if`, a condition, and a colon — followed by an indented block of exactly 4 spaces.

把 `if` 想成程序提出的问题："如果这是真的呢？"语法由三个固定部分组成：关键字 `if`、一个条件、一个冒号——随后是缩进严格为 4 空格的代码块。

```
age = 18
if age >= 18:
    print("You can vote!")   # must be indented! / 必须缩进!
```

The colon is mandatory; missing it is a `SyntaxError`. The indentation is mandatory; missing it is an `IndentationError`.

冒号必不可少，漏写是 `SyntaxError`；缩进必不可少，漏写是 `IndentationError`。

Adding an alternative: `else` 加上另一条路：`else`

```
temperature = 30
if temperature == 30:
    print("Hot")
else:
    print("Not hot")
```

> **THE CLASSIC BEGINNER MISTAKE** 经典新手错误
>
> Single = assigns; double == compares. `if temperature = 30:` is a syntax error — Python refuses the confusion between "set it" and "ask about it".
>
> 单等号是赋值，双等号是比较。`if temperature = 30:` 是语法错误——Python 拒绝"设置"与"询问"之间的混淆。

6.2 Comparison and Logical Operators

比较与逻辑运算符

Conditions are expressions that produce `True` or `False`. Comparisons build them:

条件是产生 `True` 或 `False` 的表达式。比较运算用来构造它们：

```
height = 175
print(height > 160)   # True
print(height == 170)  # False

bus_height_limit = 200
luggage_height = 210
if luggage_height > bus_height_limit:
    print("Cannot take bus")    # 行李超高，不能上车
```

Logical operators combine them:

逻辑运算符负责组合：

- `and`: both sides must be True.

  `and`：两边都必须为真。

- `or`: at least one side must be True.

  `or`：至少一边为真。

- `not`: reverses the boolean.

  `not`：取反。

```
is_raining = True
has_umbrella = False

if is_raining and has_umbrella:
    print("Go outside")     # 有伞才出门
else:
    print("Stay home")
```

6.3 Multi-condition Chains

多条件分支

When there are more than two mutually exclusive outcomes, use `elif` ("else if"). Python tests the conditions top to bottom and runs **only the first branch that is True**, skipping everything after it.

当结果多于两种且互斥时，使用 `elif`（"否则如果"）。Python 自上而下逐个测试条件，只执行第一个为真的分支，其余全部跳过。

```
age = 15

if age < 13:
    print("Child ticket")     # 儿童票
elif age < 18:
    print("Teen ticket")      # 青少年票
else:
    print("Adult ticket")     # 成人票
```

A complete example: ticket pricing 完整示例：票价系统

```
age = 25
is_student = False
has_membership = True

if age < 5:
    print("Free ticket")              # 免票
elif age < 18:
    print("Child ticket")             # 儿童票
elif is_student:
    print("Student discount")         # 学生折扣
elif has_membership:
    print("Membership discount")      # 会员折扣
else:
    print("Regular ticket")           # 原价票
```

THINK ABOUT IT 想一想

1. What does a 19-year-old student *without* membership get?

19 岁的学生、没有会员，得到什么？ 2. What does a 19-year-old student *with* membership get?

Answers: (1) `Student discount` — the first two conditions fail, `is_student` fires. (2) Still `Student discount`! The `elif` chain executes **only the first** True branch; once the student discount triggers, the

membership check is never reached. (3) Restructure the code: use independent `if` statements to accumulate a discount, or combine conditions such as `if is_student and has_membership:`.

答案：(1) `Student discount`——前两个条件不成立，`is_student` 命中。(2) 仍然是 `Student discount`！`elif` 链只执行第一个为真的分支；学生折扣一旦触发，会员检查根本不会执行。(3) 重构代码：用相互独立的 `if` 累加折扣，或组合条件，如 `if is_student and has_membership:`。

6.4 Nested Conditions

嵌套条件

An `if` can live inside another `if`. Each level adds 4 spaces. For readability, keep nesting to three levels at most.

`if` 可以嵌在另一个 `if` 里。每层多缩进 4 空格。为了可读性，嵌套最多不要超过三层。

```
age = 20
is_student = True

if age < 18:
    print("50% discount")
else:
    # level 2 / 第二层
    if is_student:
        print("30% discount")
    else:
        print("No discount")
```

6.5 Truth Value Testing

真值测试

Any value can be tested in a boolean context. The common **false values** are `False`, `None`, `0`, `""` (empty string) and `[]` (empty list); almost everything else is true.

任何值都可以放进布尔语境测试。常见的假值有 `False`、`None`、`0`、`""`（空字符串）与 `[]`（空列表）；其余几乎都为真。

```
name = ""
if not name:                    # empty string acts as False / 空字符串相当于 False
    print("Name is required!")  # 必须输入名字!
```

6.6 The Short-form Conditional

简写条件表达式

For simple two-way assignments, Python offers the ternary expression. It is compact but harder to read for beginners — learn it, recognize it, and prefer the full form while your eye is training.

对于简单的二选一赋值，Python 提供三元表达式。它紧凑，但对新手不友好——学会识别它，在眼力练成之前优先写完整形式。

```
temperature = 28

# short form / 简写
message = "Hot" if temperature > 25 else "OK"

# preferred for beginners / 推荐新手写法
if temperature > 25:
    message = 'Hot'
else:
    message = 'OK'
```

6.7 Keeping Conditions Readable

让条件保持可读

Two habits keep long conditions maintainable:

两个习惯让复杂条件保持可维护：

```
# wrap long conditions and break the line / 长条件用括号换行
if (temperature > 100
        and humidity < 70
        and is_sunny):
    print("Heat warning!")

# better: name the sub-conditions / 更好：给子条件命名
is_positive_even = (num > 0 and num % 2 == 0)
within_limit = (num < 100)
valid_user = (name != "admin")

if is_positive_even and within_limit and valid_user:
    print("Access granted.")
```

KEY POINTS 本章要点

- `if/elif/else`: colon + 4-space block; only the first True branch runs.

  `if/elif/else`：冒号 + 4 空格块；只执行第一个为真的分支。

- `=` assigns, `==` compares.

= 赋值，== 比较。

- `and`/`or`/`not` combine conditions; `elif` order controls outcomes.

  `and`/`or`/`not` 组合条件；`elif` 顺序决定结果。

- False values: `False`, `None`, `0`, `""`, `[]`.

  假值：`False`、`None`、`0`、`""`、`[]`。

- Name sub-conditions instead of writing one long line.

  给子条件命名，别写超长一行。

EXERCISES 练习

- Write `traffic_light.py`: ask for a color and print STOP / SLOW DOWN / GO / INVALID COLOR. Test all four paths.

  编写 `traffic_light.py`：询问颜色并输出 STOP / SLOW DOWN / GO / INVALID COLOR，测试全部四种路径。

- Build the ticket system above and answer the three "Think About It" questions in your report with evidence.

  实现上面的票价系统，并在报告中用运行结果回答"想一想"中的三个问题。

- Write `grade_system.py` that loops until the user types `q`, and maps scores to grades A/B/C/D.

  编写 `grade_system.py`：循环直到输入 `q`，并把分数映射为 A/B/C/D。

Going in Loops / 循环结构

> *Write the exit first. Everything else in the loop is detail.*
>
> 先写好出口，循环里其余的都是细节。

LEARNING OBJECTIVES 学习目标

- Write `while` loops whose conditions are always updated.
  编写条件总会被更新的 `while` 循环。
- Iterate over sequences with `for`, and generate ranges with `range()`.
  用 `for` 遍历序列，用 `range()` 生成数列。
- Exit early with `break`, skip steps with `continue`, and use the loop `else`.
  用 `break` 提前退出、用 `continue` 跳过本轮，并使用循环的 `else`。
- Combine loops safely, including nested loops.
  安全地组合循环，包括嵌套循环。

7.1 Why Loops Exist

循环为何存在

Control flow matters for three reasons: programs must make decisions, repeat tasks efficiently, and adapt to real situations. Chapter 6 covered decisions; loops are the repetition machine.

控制流重要有三个原因：程序要做决策、要高效地重复任务、要适应真实场景。第 6 章讲了决策；循环就是那台重复的机器。

A loop repeats a block of code as long as a condition remains true — like a persistent alarm clock that keeps ringing until you get up.

只要条件保持为真，循环就会重复执行一段代码——像一个一直响到你起床为止的闹钟。

```
import time
runs_remaining = 3

while runs_remaining > 0:
```

```
    time.sleep(1)   # suppose you are a superman: 400m in 1s / 假设你是超人：1 秒 400
米
    print("Morning run #" + str(4 - runs_remaining) + " completed!")
    runs_remaining = runs_remaining - 1

print("All runs finished!")
```

THE INFINITE LOOP 死循环

```
temperature = 25
while temperature > 20:
    print("AC running")
    # forgot to decrease temperature! / 忘了降低温度！
```

A loop whose condition never becomes false runs forever, freezing your terminal. Always design the termination *before* writing the body — set the initial value, define the limit, and update the control variable on every pass. `Ctrl + C` kills a runaway loop in the terminal.

条件永远不为假的循环会永远运行，冻住你的终端。写循环体之前先设计终止条件——设初值、定上限、每轮更新控制变量。终端里 `Ctrl + C` 可以杀掉失控的循环。

7.2 Safe Loop Writing

安全写循环

Write the skeleton first, fill in the body second:

先写骨架，后填内容：

```
# 1. set the initial value / 设定初值
counter = 0
# 2. define the termination limit / 定义终止上限
MAX_ATTEMPTS = 3

while counter < MAX_ATTEMPTS:
    print("Attempt:", counter + 1)
    # 3. ALWAYS update the control variable / 永远记得更新控制变量
    counter = counter + 1
```

Two classic control patterns cover most needs:

两种经典控制模式覆盖绝大多数需求：

```
# flag-controlled loop / 标志控制循环
running = True
while running:
    user_input = input("Continue? (y/n): ")
    if user_input == "n":
        running = False

# counter-controlled loop / 计数器控制循环
```

```
attempts = 3
while attempts > 0:
    # do something / 做点什么
    attempts = attempts - 1
```

Worked example: the vending machine 实例：自动售货机

```
coins = 0
while coins < 5:
    insert_coins = int(input("Insert coin (1-3): "))
    coins = coins + insert_coins
    print("Total inserted: " + str(coins))
print("Here's your drink!")
```

THINK ABOUT IT 想一想

How can we limit each insertion to 1–3 coins, and return the change afterwards?

如何限制每次只能投 1–3 枚硬币？投多了如何找零？

Validate the input *before* adding it, and compute the change after the loop:

在累加*之前*验证输入；循环结束后计算找零：

```
if 1 <= insert_coins <= 3:
    coins = coins + insert_coins
else:
    print("Invalid coin! Please insert 1, 2, or 3.")
change = coins - 5
if change > 0:
    print("Here is your change: " + str(change))
```

7.3 The `for` Loop

`for` 循环

A `for` loop iterates over a sequence — a list, a string, a range — visiting each item once. Think of checking a shopping list item by item.

`for` 循环遍历一个序列——列表、字符串或范围——每个元素访问一次。就像逐项核对购物清单。

```
groceries = ["apple", "milk", "bread"]
for item in groceries:
    print("Don't forget to buy:", item)
```

The `range()` function `range()` 函数

`range()` generates a sequence of numbers and takes one, two or three arguments:

`range()` 生成数字序列，可接受一、二或三个参数：

```
for num in range(5):        # one arg: 0,1,2,3,4 (5 excluded / 不含 5)
    print(num)

for num in range(2, 5):     # two args: start, end / 起点，终点
    print(num)              # 2,3,4

for num in range(0, 10, 2): # three args: start, end, step / 起点，终点，步长
    print(num)              # 0,2,4,6,8
```

Index and value together 索引与值同时获取

```
word = "hello"
for index, letter in enumerate(word):
    print("Character at position " + str(index) + ": " + letter)
```

`enumerate()` hands you the position and the character at the same time — no manual counter needed.

`enumerate()` 同时给你位置和字符——不需要手工计数器。

7.4 `break`, `continue` and `else`

`break`、`continue` 与 `else`

Three keywords give you fine control inside any loop:

三个关键字提供循环内的精细控制：

`break` — the emergency exit 紧急出口

`break` leaves the *entire* loop immediately.

`break` 立即跳出*整个*循环。

```
passwords = ["123", "admin", "letmein"]
for pwd in passwords:
    if pwd == "admin":
        print("Security alert! Access blocked.")
        break                   # loop ends here / 循环到此结束
    print("Checking:", pwd)
```

`continue` — skip this round 跳过本轮

`continue` abandons the rest of the *current* iteration and returns to the top of the loop.

`continue` 放弃*本轮*剩余部分，回到循环顶部。

```
for num in range(10):
    if num % 2 != 0:    # odd? / 奇数?
        continue        # skip to next / 跳到下一个
    print(num, "is even")
```

`else` — a loop that finished normally 正常跑完的循环

A unique Python feature: the `else` block runs only if the loop completed **without** hitting `break`.

Python 独有特性：只有当循环没有触发 `break` 而正常完成时，`else` 块才会执行。

```
numbers = [2, 4, 6]
for n in numbers:
    if n % 2 != 0:
        print("Found an odd number!")
        break
else:
    print("All numbers are even!")   # runs because no break happened / 未触发 break 才执行
```

7.5 Nested Loops

嵌套循环

A loop inside a loop: the inner loop runs completely for every single pass of the outer loop. Indentation keeps the levels straight.

循环里再套循环：外层每走一步，内层完整跑一遍。缩进负责区分层级。

```
for i in range(1, 4):      # outer: rows / 外层：行
    for j in range(1, 4):  # inner: columns / 内层：列
        print(str(i) + 'x' + str(j) + '=' + str(i*j), end="\t")
    print()                # newline after each row / 每行结束换行

1x1=1   1x2=2   1x3=3
2x1=2   2x2=4   2x3=6
3x1=3   3x2=6   3x3=9

for x in range(2):        # level 1 / 第一层
    for y in range(2):    # level 2 / 第二层
        if x == y:        # level 3 / 第三层
            print("Match at " + str(x) + ',' + str(y))
    print("----")         # back to level 1 / 回到第一层
```

7.6 Putting It Together: The Guessing Game

综合实战：猜数字游戏

Everything from this part — conditionals, loops, counters, `break` — combines into one complete program.

本部分的一切——条件、循环、计数器、`break`——组合成一个完整程序。

```
import random
target = random.randint(1, 100)
attempts = 0
max_attempts = 5
```

```
print("Welcome! You have 5 tries.")

while attempts < max_attempts:
    guess = int(input("Guess (1-100): "))
    attempts = attempts + 1

    if guess == target:
        print("Correct!")
        break                       # exit early on a win / 猜中提前退出
    elif guess > target:
        print("Too high")
    else:
        print("Too low")

# outside the loop: check how it ended / 循环外：检查结局
if guess == target:
    print("Victory in " + str(attempts) + " tries!")
else:
    print("Game over! The number was " + str(target))
```

KEY POINTS 本章要点

- `while` repeats under a condition; update the control variable every pass.

  `while` 按条件重复；每轮更新控制变量。

- Design termination first: initial value, limit, update.

  先设计终止：初值、上限、更新。

- `for` visits each item of a sequence; `range(start, end, step)` generates numbers.

  `for` 遍历序列元素；`range(start, end, step)` 生成数字。

- `break` exits the whole loop, `continue` skips one round, `else` fires on normal completion.

  `break` 退出整个循环，`continue` 跳过一轮，`else` 在正常完成时执行。

- Nested loops: the inner loop completes for each outer pass.

  嵌套循环：外层每走一步，内层跑完整遍。

EXERCISES 练习

- Write `morning_run.py` and `loop_practice.py` exactly as in this chapter, and explain the three `range()` usages in your report.

  按本章原样编写 `morning_run.py` 与 `loop_practice.py`，并在报告中解释 `range()` 的三种用法。

- Complete the vending machine with input validation and change.

补全自动售货机的输入校验与找零。

- Write `loop_control.py` demonstrating `break`, `continue` and the loop `else` with your own data.
  编写 `loop_control.py`，用自己的数据演示 `break`、`continue` 与循环 `else`。
- Write `nested_loops.py` printing a 3x3 multiplication table.
  编写 `nested_loops.py`，打印 3x3 乘法表。
- Extend the guessing game: track the best (smallest) number of attempts across multiple rounds.
  扩展猜数字游戏：跨多轮记录最佳（最少）尝试次数。

PART III

Abstraction / 抽象

Abstraction is the art of naming things so you can stop thinking about them. Part II taught the machine's logic; Part III teaches the programmer's: wrap repeated work into functions, split code into files called modules, and borrow finished tools from the standard library.

抽象是一门命名艺术——给东西起好名字，从此不必再惦记它的细节。第二部分讲的是机器的逻辑；第三部分讲的是程序员的逻辑：把重复的工作包进函数，把代码拆进名为模块的文件，并从标准库借来现成的工具。

Functions / 函数

> *If you have written the same three lines twice, you are ready for a function.*
>
> 同样三行代码写到第二遍，就到了写函数的时候。

LEARNING OBJECTIVES 学习目标

- Define functions with and without parameters, and call them.
  定义带参与无参函数并调用。
- Return single and multiple values, and unpack them.
  返回单个与多个返回值，并解包。
- Use positional, keyword and default arguments.
  使用位置参数、关键字参数与默认参数。
- Explain local vs global scope and use `global` correctly.
  解释局部与全局作用域，正确使用 `global`。
- Name functions professionally.
  专业地为函数命名。

8.1 What a Function Is

函数是什么

Think about a microwave. To make popcorn you do not set the power to 80% and the timer to 3 minutes by hand — you press the "Popcorn" preset. In programming, **functions** are exactly those preset buttons: named blocks of code that do one specific job, ready to be "pressed" whenever needed.

想想微波炉。爆爆米花时你不会手动把火力调到 80%、时间设为 3 分钟——你按"爆米花"预设按钮。在编程中，函数正是这样的预设按钮：一段有名字、专做一件事的代码，随时可以"按下"（调用）。

```
def start_microwave(preset):
    if preset == "popcorn":
```

```
        print('Heating for 180 seconds...')
    elif preset == "beverage":
        print('Heating for 45 seconds...')

start_microwave("popcorn")   # press the button / 按下按钮
```

The problem functions solve 函数解决的问题

Without functions, every repetition must be typed out — and every change must be edited in every place:

没有函数时，每次重复都要重新输入——每处修改都要改遍所有位置：

```
# making 3 sandwiches, the hard way / 用笨办法做三个三明治
print("1. Get bread")
print("2. Add filling")
print("3. Close sandwich")
print("------")
# ... and the same again, and again ...
```

This violates the core rule of programming: **DRY — Don't Repeat Yourself**. Adding "cut in half" to the recipe would mean editing three places. With a function, you edit one:

这违反了编程的核心原则：**DRY——不要重复自己**。往食谱里加"切成两半"就得改三处。

有了函数，只改一处：

```
def make_sandwich(filling):
    print("Making " + str(filling) + " sandwich:")
    print("1. Get bread")
    print("2. Add " + str(filling))
    print("3. Close sandwich")
    print("------")

make_sandwich("ham")
make_sandwich("cheese")
```

8.2 Anatomy of a Function

函数的解剖

A function is like a small factory: receive ingredients, follow steps, ship a result.

函数像一家小工厂：接收原料、按步骤加工、交付成品。

```
def magic_oven(ingredient):              # header: def + name + parameters / 函数头
    output = str(ingredient) + " cookie" # body: indented code / 函数体
    return output                        # hand back the result / 交还结果
```

Three essential parts:

三个必要部分：

1. The `def` statement with the function name and its parameters, ending in a mandatory colon.
   `def` 语句加上函数名与参数，以必须的冒号结尾。
2. The indented body that performs the work.
   缩进的函数体，负责干活。
3. An optional `return` that hands back a value.
   可选的 `return`，交还一个值。

```
def circle_area(radius):
    area = 3.14 * radius ** 2
    return area

pizza_area = circle_area(30)          # save the result for later / 保存结果备用
pizza_price = pizza_area * 0.02
print("Pizza price is: $" + str(pizza_price))
```

Print and return are not the same thing: `print` shows a value to a human and gives nothing back to the program; `return` hands the value to the calling code so it can be stored and used. A function that only prints cannot participate in calculations.

打印和返回不是一回事：`print` 把值展示给人看，却不给程序留下任何东西；`return` 把值交回调用方，供其存储使用。只会 print 的函数无法参与计算。

Common syntax mistakes 常见语法错误

```
def bad_function()        # missing colon / 缺冒号
    print("Oops")

def messy_function():
print("No indent!")       # missing indentation / 缺缩进
```

8.3 Parameters in Depth

深入参数

A **parameter** is a placeholder in the function *definition*; an **argument** is the real value supplied at *call time*. Keeping the two words straight makes error messages much easier to read.

参数（**parameter**）是函数*定义*中的占位符；实参（**argument**）是*调用*时提供的真实值。分清这两个词，报错信息会好读得多。

Positional vs keyword arguments 位置参数与关键字参数

By default Python matches arguments by position. You can also name them explicitly, which makes calls readable and order-proof.

默认按位置匹配实参。也可以显式指名，让调用更可读、顺序不再敏感。

```
def make_pizza(size, toppings):
    print("Making " + str(size) + "cm pizza with " + str(toppings))

make_pizza(30, "mushrooms")                    # positional / 位置
make_pizza(toppings="cheese", size=20)         # keyword / 关键字
make_pizza(["pepperoni"], 25)                  # legal but confusing / 合法但困惑
```

Multiple parameters 多参数

```
def flexible_greet(name, greeting):
    print(str(greeting) + ' ' + str(name) + '!')

flexible_greet("Charlie", "Good morning")
flexible_greet("Diana", "Ni hao")
```

The order matters: `(greeting, name)` and `(name, greeting)` produce different sentences.

顺序很重要：`(greeting, name)` 与 `(name, greeting)` 说出的是不同的话。

Default parameters 默认参数

A parameter can carry a default value, used whenever the caller omits the argument:

参数可以带默认值，调用者省略该实参时自动启用：

```
def calculate_shipping(weight, base_fee=5.0):
    price = weight * 1.2 + base_fee
    return price

print(calculate_shipping(3.5))       # uses default / 使用默认 5.0
print(calculate_shipping(2.0, 4.0))  # overrides it / 覆盖为 4.0
```

8.4 Returning Multiple Values

返回多个值

Most languages return exactly one thing. Python quietly packs several values into a **tuple** — and the caller can unpack them in one line.

多数语言只能返回一个值。Python 会悄悄把多个值打包成元组——调用方一行即可解包。

```
def box_calculator(length, width, height):
    volume = length * width * height
```

```
    surface = 2 * (length*width + length*height + width*height)
    return volume, surface           # a tuple / 一个元组

vol, surf = box_calculator(2, 2, 2)  # unpack directly / 直接解包
print("Perfect cube volume: " + str(vol))
```

Worked example: the BMI calculator 实例：BMI 计算器

```
def calculate_bmi(weight, height):
    """Calculate Body Mass Index / 计算身体质量指数"""
    bmi = weight / (height ** 2)
    return bmi

print("BMI:", calculate_bmi(70, 1.75))   # about 22.86 / 约 22.86
```

THINK ABOUT IT 想一想

1. Given BMI and weight, how do we recover height?

已知 BMI 与体重，如何反推身高？ 2. What is the difference between `x**1/2` and `x**(1/2)`?

Answers: from $BMI = W/H^2$ we get $H = \sqrt{W/BMI}$, written in Python as `(weight / target_bmi) ** 0.5`. Question 2 is pure operator precedence: `x**1/2` means `(x**1)/2` — it cuts `x` in half! — while `x**(1/2)` computes the square root. And verification is a *round-trip test*: `calculate_bmi(70, 1.75)` gives about 22.86, and `(70/22.86)**0.5` must give back 1.75.

答案：由 $BMI = W/H^2$ 得 $H = \sqrt{W/BMI}$，Python 写作 `(weight / target_bmi) ** 0.5`。第二题是纯粹的运算顺序问题：`x**1/2` 表示 `(x**1)/2`——把 x 减半！而 `x**(1/2)` 才是开平方。验证方法是*往返测试*：`calculate_bmi(70, 1.75)` 约 22.86，则 `(70/22.86)**0.5` 必须还原出 1.75。

8.5 Variable Scope

变量作用域

Variables created inside a function are **local** — they exist only there. Variables created outside all functions are **global**.

函数内部创建的变量是**局部变量**——只在该函数内存在。在所有函数之外创建的是**全局变量**。

```
temperature = 25   # global / 全局

def adjust_temp():
    temperature = 18   # a NEW local variable / 一个新的局部变量
```

```
    print("Inside function: " + str(temperature) + "C")

adjust_temp()
print("Outside function: " + str(temperature) + "C")   # still 25! / 仍是 25!
```

Reading a global inside a function is fine. **Changing** one requires the `global` keyword, which tells Python: do not create a local copy — use the outside box.

在函数内*读取*全局变量没问题；要*修改*它则必须用 `global` 关键字，告诉 Python：别建局部副本，去用外面那个盒子。

```
score = 0

def update_points():
    global score
    score = score + 10    # now modifies the global / 现在修改的是全局变量
```

> **DESIGN ADVICE** 设计建议
>
> Functions that quietly rewrite global variables are hard to test and debug. Prefer passing values in as parameters and handing results back with `return` — keep the global keyword for genuine exceptions.
>
> 悄悄改写全局变量的函数难以测试和调试。优先用参数传入、用 `return` 交回结果；`global` 留给真正的例外。

8.6 Naming Functions

函数命名

A good function name announces exactly what it does.

好的函数名能准确宣告它做什么。

```
# good / 好
def calculate_tax(income): ...
def get_user_profile(user_id): ...
def convert_to_fahrenheit(celsius): ...

# bad / 差
def tax(inc): ...         # calculating? paying? / 在算税还是交税？
def user(id): ...         # verb missing / 缺动词
def temp_conv(c): ...     # cryptic abbreviation / 缩写难猜
```

Checklist: `snake_case`, start with a verb, at most three words.

清单：`snake_case`、以动词开头、不超过三个单词。

KEY POINTS 本章要点

- Functions are named, reusable blocks: def + parameters + body + optional return.

  函数是有名、可复用的代码块：def + 参数 + 函数体 + 可选 return。

- DRY: wrap repeated code once, call it many times.

  DRY：重复代码封装一次，反复调用。

- `return` hands values back; `print` only shows them.

  `return` 交回值；`print` 只是展示。

- Keyword and default arguments make calls readable and flexible.

  关键字参数与默认参数让调用可读而灵活。

- Local variables live and die inside the function; `global` is the escape hatch.

  局部变量生于函数、死于函数；`global` 是逃生门。

- Name functions with a verb, in `snake_case`.

  用动词、`snake_case` 命名函数。

EXERCISES 练习

- Write `sandwich_function.py` and `flexible_sandwich.py` from this chapter and show both outputs in your report.

  按本章编写 `sandwich_function.py` 与 `flexible_sandwich.py`，在报告中展示两种输出。

- Write `circle_calculator.py` with a `circle_area(radius)` function used to price a 30 cm pizza at 0.02 per square cm.

  编写 `circle_calculator.py`：实现 `circle_area(radius)`，并按每平方厘米 0.02 为 30 厘米披萨定价。

- Write `shipping_calculator.py` and `box_calculator.py` and verify the cube case by hand.

  编写 `shipping_calculator.py` 与 `box_calculator.py`，并手工验算立方体情形。

- Write `bmi_calculator.py` plus the height-recovery function, and prove correctness with a round-trip test.

  编写 `bmi_calculator.py` 与反推身高的函数，并用往返测试证明正确性。

- Write `scope_demo.py` and `global_demo.py`; explain in two sentences what each prints and why.

> 编写 `scope_demo.py` 与 `global_demo.py`；各用两句话解释输出及其原因。

Modules and the Standard Library / 模块与标准库

> *A module is a toolbox with a name on it.*
>
> 模块就是一只写名字的工具箱。

LEARNING OBJECTIVES 学习目标

- Organize your own code into modules and import them.
  把自己的代码整理成模块并导入使用。
- Choose among the three import styles and know their trade-offs.
  在三种导入方式之间选择，并清楚各自的取舍。
- Use `math`, `random`, `time`, `datetime`, `os` and `shutil` from the standard library.
  使用标准库中的 `math`、`random`、`time`、`datetime`、`os`、`shutil`。
- Explore libraries on your own with `help()` and `dir()`.
  用 `help()` 与 `dir()` 自主探索库。

9.1 From Functions to Modules

从函数到模块

If functions are preset buttons, **modules** are the whole appliance. A module is simply a Python file — a `.py` file — containing functions and variables that you want to reuse across programs.

如果说函数是预设按钮，模块就是整台机器。模块就是一个 Python 文件——`.py` 文件——里面装着你想在多个程序间复用的函数和变量。

Imagine a game. Instead of writing all the sound code inside the main file, you can `import` a pre-made module:

想象一个游戏。你可以 `import`（导入）一个现成的模块，而不必把所有音效代码都塞进主文件：

```
# sound_module.py (a separate file / 一个单独的文件)
def play_jingle():
    print("Happy tune playing!")

# main.py (your program / 你的程序)
import sound_module
sound_module.play_jingle()
```

Building your own module 构建自己的模块

Create a file named `kitchen.py`:

创建一个名为 `kitchen.py` 的文件：

```
# kitchen.py
def microwave_beep():
    print("BEEP! BEEP! BEEP!")

def oven_timer():
    print("DING! Food is ready!")
```

In any other file in the *same folder*, these tools are now importable:

在*同一文件夹*下的任何其他文件中，这些工具现在都可以导入了：

```
# use_kitchen.py
import kitchen

kitchen.microwave_beep()
kitchen.oven_timer()
```

Note the dot: `kitchen.microwave_beep()` means "the `microwave_beep` inside the `kitchen` toolbox". The dot is how Python navigates toolboxes.

注意那个点：`kitchen.microwave_beep()` 意思是"kitchen 工具箱里的 microwave_beep"。

点号就是 Python 在工具箱之间穿行的方式。

9.2 Three Import Styles

三种导入方式

Style	Code	Best when
Full import 完整导入	`import math`	Prevents name conflicts; large libraries 避免命名冲突；大型库
Specific import 特定导入	`from random import randint`	You need only 1–2 functions 只需一两个函数

Alias import 别名导入	`import pandas as pd`	Frequently used, long names 常用且名字很长

```
import math                          # full: must write math. every time
print("2^5 = " + str(math.pow(2, 5)))

from random import randint           # specific: no prefix needed
print("Dice: " + str(randint(1, 6)))

import pandas as pd                  # alias: short and standard
```

Styles can be mixed, and aliasing works with `from` too:

各种风格可以混用，`from` 也能配别名：

```
from math import sqrt as sq
print("Square root of 121: " + str(sq(121)))
```

> **NAME COLLISIONS** 命名冲突
>
> `from X import *` pours every public name of module X into your file. If you also defined a function called `sqrt`, one of the two silently wins. Prefer explicit imports — three extra characters buy you a whole class of debugging sessions.
>
> `from X import *` 会把 X 的所有公开名字倒进你的文件。如果你自己也定义了 `sqrt`，其中一个会悄悄获胜。请用显式导入——多打三个字符，省掉一整类调试。

9.3 The Standard Library

标准库

Python ships with a "batteries included" standard library — hundreds of modules installed with the interpreter itself. No download needed.

Python 自带"装好电池"的标准库——数百个模块随解释器一起安装，无需下载。

Module	What it does
`math`	Mathematical functions: `sqrt`, `pi`, `pow` 数学函数：`sqrt`、`pi`、`pow`
`random`	Random numbers and choices 随机数与随机选取

`time`	Pausing, timestamps 暂停、时间戳
`datetime`	Dates, times, arithmetic on both 日期、时间及其运算
`os`	Operating system interface: files, folders 操作系统接口：文件与文件夹
`shutil`	High-level file operations: copy, move 高级文件操作：复制、移动

math math 库

```
import math

a = 3
b = 4
c = math.sqrt(a**2 + b**2)
print("The hypotenuse is: " + str(c))    # 5.0
```

random random 库

```
from random import choice

participants = ["Alice", "Bob", "Charlie", "Diana"]
winner = choice(participants)
print("Congratulations " + winner + "!")
```

time time 库

```
import time

print("Starting countdown:")
for i in range(5, 0, -1):
    print(i)
    time.sleep(1)          # pause 1 second / 暂停 1 秒
print("Blast off!")
```

datetime and timedelta datetime 与 timedelta

```
import datetime

now = datetime.datetime.now()
print("Current time: " + str(now))

future_date = now + datetime.timedelta(days=3)
print("Date after 3 days: " + str(future_date.date()))
```

`timedelta` accepts `days`, `hours`, `minutes`, `seconds` and `weeks` — time arithmetic without manual conversion.

`timedelta` 接受 `days`、`hours`、`minutes`、`seconds`、`weeks`，无需手工换算的时间运算。

9.4 Namespaces

命名空间

Think of each module as a separate toolbox. If your garage toolbox and your kitchen toolbox both contain a hammer, you must say which one you mean.

把每个模块想成一只独立的工具箱。如果车库工具箱和厨房工具箱各有一把锤子，你必须说明你要哪把。

```
import math
import mymath        # a hypothetical custom module / 假设的自制模块

math.sqrt(16)        # the hammer from the math toolbox / math 工具箱里的锤子
mymath.sqrt(16)      # the hammer from yours / 你自己工具箱里的锤子
```

Namespaces are the reason the full-import style is safest: every name stays inside its toolbox, and nothing ever collides silently.

这就是完整导入最安全的原因：每个名字都留在自己的工具箱里，永远不会无声相撞。

9.5 Exploring on Your Own

自主探索

Two built-in functions turn the standard library into a readable encyclopedia:

两个内置函数能把标准库变成一部可读的百科全书：

```
help('modules')    # list every standard module / 列出所有标准模块
help(math)         # manual for one module / 单个模块的说明书
dir(math)          # names inside it / 里面的名字清单
```

Professional programmers do not memorize libraries — they read `help()` output, official documentation at docs.python.org/3/library, and search PyPI.org when they need something new. Standing on the shoulders of giants is a job skill.

专业程序员并不背诵库——他们读 `help()` 输出、读 docs.python.org/3/library 的官方文档，需要新东西时去 PyPI.org 搜索。"站在巨人的肩膀上"本身就是一项职业技能。

KEY POINTS 本章要点

- A module is a `.py` file; a package is a folder of modules; a library is the functional whole.

  模块是 `.py` 文件；包是模块的文件夹；库是功能整体。

- Import fully, specifically, or with an alias — never `import *`.

  完整、特定或别名导入——永远不要 `import *`。

- Standard library essentials: `math`, `random`, `time`, `datetime`, `os`, `shutil`.

  标准库 essentials：`math`、`random`、`time`、`datetime`、`os`、`shutil`。

- The dot navigates namespaces and prevents silent collisions.

  点号在命名空间间穿行，防止无声冲突。

- `help()`, `dir()` and the official docs are your self-study toolkit.

  `help()`、`dir()` 与官方文档是你的自学工具。

EXERCISES 练习

- Create `kitchen.py` and `use_kitchen.py`; then write your own `cooking_module.py` with a `prepare_dish(dish_name)` function and call it twice.

  创建 `kitchen.py` 与 `use_kitchen.py`；再自创 `cooking_module.py`，含 `prepare_dish(dish_name)` 函数并调用两次。

- Write `math_practice.py`: compute the hypotenuse of a 3-4 triangle, the area of a radius-5 circle with `math.pi`, and `math.pow(2, 3)`.

  编写 `math_practice.py`：计算 3-4 三角形斜边、半径 5 圆的面积（用 `math.pi`）以及 `math.pow(2, 3)`。

- Write `countdown.py` using `time.sleep`, then a random lottery picker using `randint` and `choice`.

  编写用 `time.sleep` 的 `countdown.py`，再用 `randint` 与 `choice` 写一个随机抽奖器。

- Write `stdlib_practice.py` combining `datetime`, `random` and `os`: print the formatted current time, a random password, and create a folder if missing.

  编写 `stdlib_practice.py`：组合 `datetime`、`random` 与 `os`，打印格式化当前时间、生成随机密码、按需创建文件夹。

- Run `help('modules')` and pick one module never mentioned in this book; write five lines about

what it offers.

运行 `help('modules')`，挑一个本书没提过的模块，用五行文字介绍它的功能。

PART IV

Data Structures / 数据结构

One variable holds one thing. Real problems hold thousands of things: users, scores, votes, gene names. Part IV introduces the four containers of everyday Python — lists for ordered sequences, tuples for immutable records, dictionaries for key-value lookups, sets for uniqueness — and the judgement to pick the right one.

一个变量装一个东西。真实的问题却要装成千上万个东西：用户、分数、选票、基因名。第四部分介绍日常 Python 的四大容器——列表装有序序列，元组装不可变记录，字典装键值对，集合装唯一性——以及"选对容器"的判断力。

Lists and Tuples / 列表与元组

> *A list is a queue that changes its mind; a tuple is a promise that does not.*
>
> 列表是会改主意的队列；元组是不改口承诺。

LEARNING OBJECTIVES 学习目标

- Create lists four ways, and access and slice their elements.
  用四种方式创建列表，访问并切片其元素。
- Modify lists with `append`, `insert`, `remove`, `pop` and `sort`.
  用 `append`、`insert`、`remove`、`pop`、`sort` 修改列表。
- Explain tuple immutability and when it is the right choice.
  解释元组的不可变性及适用场景。
- Unpack sequences, including the single-element tuple trap.
  解包序列，包括单元素元组陷阱。

10.1 What Is a List?

什么是列表?

A list is an ordered, changeable collection — like a shopping list that can hold many items.

列表是有序、可更改的集合——像一张能装很多东西的购物清单。

```
groceries = ["apples", "milk", "eggs", "bread"]
print("My shopping list: " + str(groceries))
```

Four ways to create a list 创建列表的四种方式

```
colors = ["red", "green", "blue"]          # 1. square brackets / 方括号
numbers = list((1, 2, 3))                  # 2. the list() constructor / 构造函数
squares = [x**2 for x in range(5)]         # 3. list comprehension / 列表推导式
word = list("hello")                       # 4. convert from a string / 从字符串转换
```

10.2 Accessing Elements

访问元素

Indexing works exactly as with strings — positive from the start, negative from the end — and slicing returns a new list.

索引与字符串完全一致——正向从头，负向从尾——切片返回新列表。

```
games = ["Mario", "Zelda", "Pokémon"]
print(games[0])    # Mario
print(games[-1])   # Pokémon

months = ["Jan", "Feb", "Mar", "Apr", "May"]
print(months[1:3])  # ['Feb', 'Mar']
print(months[:2])   # ['Jan', 'Feb']
print(months[3:])   # ['Apr', 'May']
```

10.3 Modifying Lists

修改列表

Adding 添加

```
todo = ["wake up"]
todo.append("brush teeth")   # add to the end / 加到末尾
todo.insert(1, "make bed")   # insert at position 1 / 插到位置 1
print(todo)                  # ['wake up', 'make bed', 'brush teeth']
```

THINK ABOUT IT 想一想

What do `todo.insert(0, 'start')`, `todo.insert(len(todo), 'end1')` and `todo.insert(-1, 'end2')` each do?

`todo.insert(0, 'start')`、`todo.insert(len(todo), 'end1')`、`todo.insert(-1, 'end2')` 各是什么效果？

`insert(0, ...)` puts an item at the very beginning. `insert(len(todo), ...)` adds at the very end — it behaves just like `append()`. `insert(-1, ...)` inserts *before* the current last item, so your item lands second from the end, not last.

`insert(0, ...)` 插到最前。`insert(len(todo), ...)` 加到最后——效果等同 `append()`。

`insert(-1, ...)` 插在当前最后一个元素*之前*，所以你的元素落在倒数第二，而不是最后。

Removing 删除

```
pets = ["cat", "dog", "hamster"]
pets.remove("dog")   # remove by value / 按值删除
last = pets.pop()    # remove and return the last item / 移除并返回最后一个
print(pets)          # ['cat']
print(last)          # 'hamster'
```

THINK ABOUT IT 想一想

If `pets = ["dog", "bird", "dog", "cat", "dog"]`, what does `pets.remove("dog")` do, and what does `pets.index('dog')` return?

若 `pets = ["dog", "bird", "dog", "cat", "dog"]`，`pets.remove("dog")` 会怎样？

`pets.index('dog')` 返回什么？

`remove()` deletes only the **first** matching value, leaving `["bird", "dog", "cat", "dog"]`. `index()` returns the position of the **first** occurrence — here `0`. To remove every occurrence you need a loop or a filter.

`remove()` 只删除第一个匹配值，剩下 `["bird", "dog", "cat", "dog"]`。`index()` 返回首次出现的位置——此处是 `0`。要删除所有匹配项，需要循环或过滤。

Sorting 排序

```
scores = [88, 92, 75, 100]
scores.sort()                  # ascending / 升序
scores.sort(reverse=True)      # descending / 降序
```

10.4 Practice: Student Grades

实战：学生成绩系统

Two parallel lists — one of names, one of scores — are the classic first data model.

两个平行列表——一份名字、一份分数——是最经典的第一版数据模型。

```
students = ["Alice", "Bob", "Charlie"]
scores = [88, 92, 75]

total = sum(scores)
average = total / len(scores)
print("Class average: " + str(average))      # 85.0
```

To rank students, zip the two lists into pairs and sort:

要给学生排名，把两个列表拼成数对再排序：

```
combined = []
i = 0
while i < len(scores):
    combined.append([scores[i], students[i]])
    i = i + 1

combined.sort()                    # sorts by score first / 先按分数排
print("Ranking:")
for one in combined:
    print(str(one[1]) + ": " + str(one[0]))
```

Add `reverse=True` to `sort()` for a leaderboard from high to low.

在 `sort()` 中加 `reverse=True` 即可得到从高到低的排行榜。

10.5 Tuples: Immutable Sequences

元组：不可变序列

A tuple is a list that has made a promise: it will never change.

元组是一个做出承诺的列表：它永远不会变。

```
# lists are mutable / 列表可变
fruits = ["apple", "banana", "cherry"]
fruits[0] = "avocado"        # fine / 没问题

# tuples are immutable / 元组不可变
colors = ("red", "green", "blue")
colors[0] = "pink"           # TypeError! / 报错!
```

List	Tuple
Created with `[]` 用 `[]` 创建	Created with `()` 用 `()` 创建
Can change after creation 创建后可修改	Frozen after creation 创建后即冻结
For collections that evolve 用于会演变的集合	For data that must not change 用于不该变的数据

Why does immutability matter? Because some things in the world do not change: geographic coordinates, RGB colors, a date's components. Protecting them with a tuple means no code — not yours, not anyone's — can quietly corrupt them.

不可变性为何重要？因为世界上有些东西不会变：地理坐标、RGB 颜色、日期的年月日。

用元组保护它们，任何代码——你的也好，别人的也好——都无法悄悄破坏它们。

```
black = (0, 0, 0)
current_position = (37.7749, -122.4194)

locations = {                        # tuples work as dict keys; lists do not!
    (40.7128, -74.0060): "New York",
    (51.5074, -0.1278): "London"
}                                    # 元组可作字典键；列表不行！
```

Tuple unpacking 元组解包

```
dimensions = (1920, 1080)
width, height = dimensions           # basic unpacking / 基础解包
print("Screen: " + str(width) + "x" + str(height))

a, b = 5, 10
a, b = b, a                          # swap without a temp variable / 无临时变量交换
```

Advanced unpacking collects the leftovers:

进阶解包负责收拢剩余部分：

```
coordinates = (45, -73, 100)
lat, lon, _ = coordinates            # _ means "ignore this" / _ 表示"忽略"

numbers = (3, 1, 4, 1, 5, 9)
first, *middle, last = numbers
print(middle)                        # [1, 4, 1, 5] — a list of the rest / 其余元素收进列表
```

THE SINGLE-ELEMENT TRAP 单元素陷阱

```
not_a_tuple = (42)         # just the number 42 in parentheses / 只是带括号的数字 42
print(type(not_a_tuple))   # <class 'int'>

proper_tuple = (42,)       # the comma makes the tuple / 逗号才造出元组
print(type(proper_tuple)) # <class 'tuple'>
```

The parentheses never made a tuple — the **comma** did. Without the comma, `(42)` is plain arithmetic grouping. And of course `proper_tuple[1]` raises `IndexError`: there is no second element.

造就元组的从来不是括号，而是逗号。没有逗号，`(42)` 只是普通的算术分组。理所当然，`proper_tuple[1]` 会抛出 `IndexError`：根本没有第二个元素。

Tuple operations 元组操作

```
my_tuple = (1, 2, 3, 2, 2)
print(len(my_tuple))       # 5
print(2 in my_tuple)       # True
print(my_tuple.count(2))   # 3
print(my_tuple.index(3))   # 2
```

Remember what is *missing*: no `append`, no `remove`, no `pop`. A tuple has promised not to change, and Python enforces the promise.

记住缺什么：没有 `append`、`remove`、`pop`。元组承诺不变，Python 负责强制兑现。

KEY POINTS 本章要点

- Lists: create with `[]`, modify with `append`/`insert`/`remove`/`pop`, order with `sort`.

  列表：`[]` 创建，`append`/`insert`/`remove`/`pop` 修改，`sort` 排序。

- `remove` and `index` affect only the first match.

  `remove` 与 `index` 只处理第一个匹配。

- Tuples are created with `()` — really with the comma — and never change.

  元组用 `()` 创建——真正起作用的是逗号——且永不变。

- Unpacking swaps variables and splits sequences: `first, *middle, last = ...`.

  解包能交换变量、切分序列：`first, *middle, last = ...`。

- Tuples make safe constants and dictionary keys.

  元组适合做安全常量与字典键。

EXERCISES 练习

- Write `list_demo.py`, `list_creation.py` and `list_access.py` covering all four creation methods and all slices shown above.

  编写 `list_demo.py`、`list_creation.py`、`list_access.py`，覆盖四种创建方式与上文全部切片。

- Write `list_add.py` and `list_remove.py`, including the three `insert` positions and the duplicate-`remove` experiment.

  编写 `list_add.py` 与 `list_remove.py`，包含三种 `insert` 位置与重复值 `remove` 实验。

- Write `student_grades.py`: compute the class average, then rank students high-to-low by score.

  编写 `student_grades.py`：计算班级平均分，再按分数从高到低排名。

- Write `tuple_demo.py`, `tuple_unpack.py` and `tuple_single.py`; document the `(42)` versus `(42,)` result.

  编写 `tuple_demo.py`、`tuple_unpack.py`、`tuple_single.py`；记录 `(42)` 与 `(42,)` 的结果差异。

- Write `tuple_return.py` with `circle_calc(radius)` returning area and circumference; print both via unpacking.

  编写 `tuple_return.py`：`circle_calc(radius)` 返回面积与周长，并用解包打印两者。

Dictionaries and Sets / 字典与集合

> *A dictionary answers questions by key; a set answers only one question: have I seen you before?*
>
> 字典按键回答问题；集合只回答一个问题：我见过你吗？

LEARNING OBJECTIVES 学习目标

- Create, update, query and delete dictionary entries safely.

  安全地创建、更新、查询、删除字典条目。

- Use `get()`, `keys()`, `values()` and `items()`.

  使用 `get()`、`keys()`、`values()`、`items()`。

- Explain set uniqueness and compute intersections, unions and differences.

  解释集合的唯一性，并计算交、并、差集。

- Combine dictionaries and lists to model structured records.

  组合字典与列表为结构化记录建模。

11.1 The Phone Book Model

电话簿模型

A **dictionary** maps keys to values — exactly like the contacts app maps a name to a phone number.

字典把键映射到值——正如通讯录把名字映射到电话号码。

```
phone_book = {
    "Alice": "555-1234",
    "Bob": "555-5678",
    "Charlie": "555-9012"
}
print(phone_book["Alice"])    # 555-1234
```

Three ways to create a dictionary 创建字典的三种方式

```
student = {"name": "Emma", "age": 20, "major": "CS"}        # curly braces / 花括号
colors = dict(red="#FF0000", green="#00FF00")               # dict() constructor
weekdays = dict([(1, "Mon"), (2, "Tue"), (3, "Wed")])       # list of tuples / 元组列表
```

11.2 Updating Dictionaries

更新字典

Assignment adds a new key or overwrites an existing one — the same syntax does both:

赋值既能添加新键，也能覆盖旧键——同一语法两用：

```
animal_sounds = {}
animal_sounds["dog"] = "Woof!"   # add / 添加
animal_sounds["cat"] = "Mew"     # add / 添加
animal_sounds["cat"] = "Meow"    # update / 更新
print(animal_sounds)             # {'dog': 'Woof!', 'cat': 'Meow'}

del animal_sounds["dog"]              # delete an entry / 删除条目
removed = animal_sounds.pop("cat")    # delete and get the value / 删除并取值
print(removed)                        # "Meow"
```

Safe access with `get()` 用 `get()` 安全访问

Direct indexing crashes when the key is missing; `get()` lets you supply a fallback instead:

直接索引在键不存在时会崩溃；`get()` 允许你提供兜底值：

```
grades = {"Math": 90, "Science": 85}
print(grades.get("History", "Subject not found"))
# Subject not found — no crash / 不崩溃
```

The methods 方法清单

```
my_dict = {"a": 1, "b": 2}
print(my_dict.keys())     # dict_keys(['a', 'b'])
print(my_dict.values())   # dict_values([1, 2])
print(my_dict.items())    # dict_items([('a', 1), ('b', 2)])
```

Power tips 高阶要点

1. Keys can be numbers, strings or tuples — anything immutable.

   键可以是数字、字符串或元组——任何不可变对象。

2. Values can be **any** type at all, including lists and other dictionaries.

   值可以是任何类型，包括列表和字典。

3. Lookup speed is extremely fast, regardless of size.

   无论多大，查找都极快。

```
mixed_example = {
    1: "Number key",
    (2, 3): "Tuple key",
    "list": ["can", "be", "value"]
}
```

11.3 Two Small Projects

两个小项目

A translator 翻译器

```
translator = {
    "hello": "hola",
    "goodbye": "adiós",
    "thank you": "gracias"
}

word = input("Enter English word: ").lower()
translation = translator.get(word, "Translation not available")
print("Spanish: " + str(translation))

if translation == "Translation not available":
    print("Please check spelling!")
    print("Current words: " + str(list(translator.keys())))
```

Note the two defensive habits: `.lower()` normalizes the input, and `get()` handles the unknown word gracefully.

注意两个防御习惯：`.lower()` 归一化输入，`get()` 优雅处理未知单词。

Movie ratings 电影评分

```
movies = {
    "Moana": 7.5,
    "Toy Story": 8.6,
    "Frozen": 8.5
}

movie = input("Enter movie title: ")
print("Rating: " + str(movies.get(movie, "Not in database")) + "/10")
```

11.4 Sets

集合

A **set** is a collection with three defining behaviors: elements are automatically **unique**, the set is **unordered** (no indexing), and it supports mathematical set operations.

集合有三个定义性行为：元素自动唯一、无序（不能索引访问）、支持数学集合运算。

```
fruits_list = ['apple', 'banana', 'apple', 'orange']
fruits_set  = {'apple', 'banana', 'apple', 'orange'}

print("List: " + str(fruits_list))   # keeps duplicates / 保留重复
print("Set:  " + str(fruits_set))    # {'apple', 'banana', 'orange'} / 自动去重
```

Deduplication is the set's most-used trick:

去重是集合最常用的招式：

```
votes = ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob']
unique_voters = set(votes)
print("Total votes: " + str(len(votes)))          # 5
print("Unique voters: " + str(len(unique_voters)))  # 3
```

Set operations 集合运算

Operation	Symbol	Question it answers
Intersection 交集	`&`	Who is in *both* groups? 谁同时在两个组里？
Union 并集	`|`	Who is in *at least one* group? 谁至少在一个组里？
Difference 差集	`-`	Who is in the first but *not* the second? 谁在第一个组却不在第二个？

```
art_students   = {'Tom', 'Alice', 'Bob', 'Eve'}
music_students = {'Alice', 'Eve', 'John'}

both = art_students & music_students
print("In both clubs: " + str(both))          # {'Alice', 'Eve'}

week1 = {'Alice', 'Bob'}
week2 = {'Bob', 'Charlie'}
print("All customers: " + str(week1 | week2)) # {'Alice', 'Bob', 'Charlie'}

available = {'red', 'blue', 'green'}
used = {'blue', 'yellow'}
print("Remaining: " + str(available - used))  # {'red', 'green'}
```

Set methods 集合方法

```
s = {'a', 'b', 'c'}
s.add('x')          # add an element / 添加元素
s.remove('b')       # remove an element / 删除元素
s.clear()           # empty the set / 清空
new_set = s.copy()  # duplicate / 复制
```

Lists also have `clear()` and `copy()` — the same tool, two containers.

列表同样有 `clear()` 与 `copy()`——同一工具，两种容器。

Worked example: counting votes 实战：统计票数

Sets answer "who ran?"; a dictionary answers "by how much?". Combining them is a classic pattern.

集合回答"有谁参选？"；字典回答"各得多少票？"。两者组合是经典模式。

```
votes = ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob', 'Alice']
print("Candidates: " + ', '.join(set(votes)))

vote_counts = {}
for candidate in votes:
    if candidate in vote_counts:
        vote_counts[candidate] += 1
    else:
        vote_counts[candidate] = 1
print(vote_counts)     # {'Alice': 3, 'Bob': 2, 'Charlie': 1}
```

11.5 Structured Records

结构化记录

The real power appears when dictionaries live inside lists: each student becomes one record.

真正的威力出现在字典住进列表之时：每个学生成为一条记录。

```
students = []

def add_student():
    name = input("Enter name: ")
    age = int(input("Enter age: "))
    major = input("Enter major: ")
    students.append({
        "name": name,
        "age": age,
        "major": major
    })
    print("Student added!")
```

Sorting records by a field uses a `key` function:

按字段排序需要 `key` 函数：

```
students.sort(key=lambda s: s["age"])   # youngest first / 按年龄升序
for s in students:
    print(s["name"] + ": " + str(s["age"]))
```

`lambda s: s["age"]` is a tiny unnamed function that extracts the sort key from each record. You will meet `lambda` formally in later courses; for now, treat it as "sort by this field".

`lambda s: s["age"]` 是一个从每条记录中提取排序键的小型无名函数。后续课程会正式讲解 `lambda`；目前把它理解为"按这个字段排序"即可。

A mutable-vs-immutable case study 可变与不可变案例

```
a = []
def add():
    a.append(1)      # modifying the list inside the function / 在函数内修改列表
add()
print(a)             # [1] — worked without global / 无需 global 即成功

a = 0
def add():
    a = a + 1        # UnboundLocalError! / 报错!
```

The difference is *assignment* versus *modification*. Assignment (`a = a + 1`) replaces the whole box, and doing that inside a function requires `global`. Modification (`a.append(1)`, `a.add(1)`, `a[1] = 1`) changes what is inside the box — the box itself is untouched, so no `global` is needed. Lists, dictionaries and sets are modifiable in place; integers and strings are not.

区别在于*赋值*与*修改*。赋值（`a = a + 1`）是换掉整个盒子，在函数内做这件事必须 `global`。修改（`a.append(1)`、`a.add(1)`、`a[1] = 1`）只是动盒子里的东西——盒子没换，所以无需 `global`。列表、字典、集合可以原地修改；整数和字符串不行。

11.6 Word Frequency Analyzer

词频分析器

We close with the classic text-mining exercise: count the top-5 words in a text.

以经典的文本挖掘练习收尾：统计文本中出现频率前五的单词。

```
text = "apple banana orange apple pear orange apple ok yes yes ok no no want"
words = text.lower().split()

word_counts = {}
for word in words:
    if word in word_counts:
        word_counts[word] += 1
    else:
        word_counts[word] = 1

# to a list of [count, word] pairs, then sort descending / 转成 [次数, 单词] 对再降序排
count_list = []
for word, count in word_counts.items():
    count_list.append([count, word])
count_list.sort(reverse=True)

print("Top 5 words:", count_list[:5])
```

Notice the design decisions: lowercase everything before counting, store counts as the *first* element so plain `sort()` orders by them, and slice `[:5]` for the leaderboard. Small modeling choices make the algorithm trivial.

注意其中的设计决策：统计前先统一小写；把次数放在数对*第一位*，让普通 `sort()` 直接按它排序；用 `[:5]` 截取榜单。恰当的建模选择能让算法变得平凡。

KEY POINTS 本章要点

- Dictionaries map keys to values; add/update with `d[k] = v`, read safely with `get()`.
  字典把键映射到值；`d[k] = v` 添加或更新，`get()` 安全读取。
- `keys()`、`values()`、`items()` expose the three views of a dictionary.
  `keys()`、`values()`、`items()` 暴露字典的三个视图。
- Sets auto-deduplicate and support `&`, `|`, `-`.
  集合自动去重，支持 `&`、`|`、`-`。
- Dict + set = counting votes; dict + list = structured records.
  字典 + 集合 = 统计选票；字典 + 列表 = 结构化记录。
- Assignment replaces the box (needs `global`); modification changes the contents (does not).
  赋值换盒子（需要 `global`）；修改动内容（不需要）。

EXERCISES 练习

- Write `dict_create.py` and `dict_modify.py` covering all three creation methods and all update/delete operations.
  编写 `dict_create.py` 与 `dict_modify.py`，覆盖三种创建方式与全部更新/删除操作。
- Write `translator.py` and `movie_rating.py`; test both an existing word and a missing one.
  编写 `translator.py` 与 `movie_rating.py`；各测试一个存在的词和一个缺失的词。
- Write `set_dedup.py` and `set_operations.py`; include the unique-letters-in-your-name program.
  编写 `set_dedup.py` 与 `set_operations.py`；包含"名字中的不重复字母"程序。
- Write `set_voting.py` combining unique candidates with per-candidate vote counts.

编写 `set_voting.py`，组合唯一候选人名单与每人票数。

- Write `student_system.py`: add two students, then sort them by age using `key=lambda s: s['age']`.

  编写 `student_system.py`：添加两名学生，再用 `key=lambda s: s['age']` 按年龄排序。

- Write `word_frequency.py` printing the top-5 words of the sample text; then run it on a paragraph you write yourself.

  编写 `word_frequency.py` 打印样例文本的前五高频词；再对自写的一段话运行。

PART V

The Real World / 与真实世界对接

So far every variable you created died with the program. Part V connects your code to the world outside the terminal: files that persist, encodings that make text portable, exceptions that keep programs alive, libraries downloaded from the internet, and tasks that run concurrently. This is where Python stops being an exercise and starts being a tool.

到目前为止，你创建的每个变量都随程序结束而消亡。第五部分把你的代码连接到终端之外的世界：持久化的文件、让文本可迁移的编码、让程序不轻易死亡的异常、从互联网下载的库，以及并发执行的任务。从这里开始，Python 不再是练习题，而是工具。

Reading and Writing Files / 文件读写

> *A file is a bottle: it stores what you pour in, long after you walk away.*
>
> 文件是一只瓶子：你倒进去的东西，在你离开很久之后仍在。

LEARNING OBJECTIVES 学习目标

- Distinguish text files from binary files, and absolute from relative paths.
  区分文本文件与二进制文件、绝对路径与相对路径。
- Open files in read, write and append modes.
  以读、写、追加三种模式打开文件。
- Read with `read()`, `readline()` and `readlines()`; write with `write()` and `writelines()`.
  用 `read()`、`readline()`、`readlines()` 读取；用 `write()`、`writelines()` 写入。
- Use the `with` statement to guarantee cleanup.
  使用 `with` 语句保证资源清理。

12.1 What Is a File?

什么是文件?

A file is a data container stored on permanent storage — a hard disk. Think of it metaphorically as a water bottle: just as a bottle stores liquid for later, a file stores data so programs can retrieve and manipulate it long after the program that wrote it has exited.

文件是存储在持久化设备（硬盘）上的数据容器。可以把它比喻成水瓶：瓶子储存液体以备后用，文件储存数据，让程序在写入它的程序退出很久之后仍能读取和操作。

Type	Examples and use
Text file 文本文件	Human-readable characters: `.txt`, `.csv`, `.py` 人类可读的字符：`.txt`、`.csv`、`.py`
Binary file 二进制文件	Computer formats: `.jpg`, `.docx`, `.exe` 计算机专用格式：`.jpg`、`.docx`、`.exe`

12.2 Paths Revisited

再谈路径

To open a file, the computer must know exactly where it is. Paths are the file's home address.

要打开文件，计算机必须知道它的确切位置。路径就是文件的家庭住址。

```
"/home/student/documents/diary.txt"   # absolute: from the root / 绝对：从根开始
"../pictures/vacation.jpg"            # relative: from where you are / 相对：从当前位置
```

12.3 Opening Files: Modes

打开文件：模式

The `open()` function takes a path and a **mode** — a single letter declaring your intention:

`open()` 函数接受路径和模式——一个声明意图的字母：

Mode	Name	Behavior
`r`	read 读	Reads existing content; fails if the file is missing 读取现有内容；文件不存在则报错
`w`	write 写	Overwrites the file entirely, or creates it 完全覆盖，或新建文件
`a`	append 追加	Adds new content to the end, keeping the old 在末尾添加，保留原内容

```
f = open("list.txt", "r")   # read / 读
f = open("new.txt", "w")    # write (overwrites!) / 写（会覆盖！）
f = open("log.txt", "a")    # append / 追加
```

THE `W` TRAP `w` 陷阱

Opening with `"w"` erases the file *at that moment*, before you write a single character. If you meant to add to a diary and used `"w"`, yesterday is gone. Double-check the mode every single time.

以 `"w"` 打开的瞬间，文件就被清空了——你一个字都还没写。本想往日记里追加却用了 `"w"`，昨天就没了。每次都要核对模式。

12.4 Reading

读取

The `read()` family offers three granularities:

`read()` 家族提供三种粒度：

```
file = open("classlist.txt", "r")

print(file.read())       # the whole file as one string / 整个文件为一个字符串
file.seek(0)             # rewind the cursor to the start / 把光标拨回开头

print(file.readline())   # one line at a time / 一次一行
file.seek(0)

print(file.readlines())  # all lines as a list of strings / 所有行组成的列表

file.close()             # always close! / 永远记得关闭!
```

`seek(0)` matters because reading consumes the file: after `read()`, the cursor sits at the end, and a second `read()` returns nothing until you rewind.

`seek(0)` 很重要，因为读取是消耗性的：`read()` 之后光标停在末尾，不回拨的话第二次 `read()` 什么也读不到。

Case: reading a class list 案例：读取班级名单

```
file = open("classlist.txt", "r")
students = file.readlines()
file.close()

for student in students:
    print("Hello " + student.strip() + "!")
```

`strip()` removes the invisible `\n` at the end of every line — without it, every greeting prints on two lines.

`strip()` 去掉每行末尾看不见的 `\n`——没有它，每句问候都会多占一行。

12.5 Writing

写入

```
# write() takes strings / write() 接收字符串
file = open("groceries.txt", "w")
file.write("Milk\n")          # \n = newline / \n 表示换行
file.write("Eggs\nBread\n")
file.close()
```

```
# writelines() takes a list of strings / writelines() 接收字符串列表
items = ["Apples\n", "Coffee\n"]
file = open("shopping.txt", "w")
file.writelines(items)
file.close()
```

Case: appending a diary 案例：追加日记

```
file = open("diary.txt", "a")       # 'a' keeps the past / 'a' 保留过去
file.write("\n2024-06-01: Learned file operations!")
file.close()
```

12.6 The `with` Statement

`with` 语句

Forgetting `close()` is not a style issue — it is a data-loss risk. If an error occurs between `open()` and `close()`, the cleanup line never runs:

忘记 `close()` 不是风格问题——是数据丢失风险。若 `open()` 与 `close()` 之间发生错误，

清理代码永远不会执行：

- Data may not be fully written to disk — the most dangerous outcome.

  数据可能没有完整写入磁盘——最危险的后果。

- File handles leak, eventually exhausting the system.

  文件句柄泄漏，最终耗尽系统资源。

- The file may stay locked for other programs.

  文件可能对其他程序保持锁定。

The `with` statement is the fix. It is a **context manager**: it guarantees cleanup no matter what happens inside.

`with` 语句就是解法。它是一个上下文管理器：无论内部发生什么，都保证清理。

```
with open('diary.txt', 'r') as f:
    content = f.read()
# the file closes here automatically — even after errors / 文件在此自动关闭——即使发生过错误
```

Multiple files at once 同时管理多个文件

```
with open('original.txt', 'r') as src:
    with open('backup.txt', 'w') as dst:
        dst.write(src.read())
```

Case: binary copy 案例：二进制复制

Binary files use the `b` variants of the modes:

二进制文件使用模式的 `b` 变体：

```
with open('cat.jpg', 'rb') as source_cat:
    with open('cat_backup.jpg', 'wb') as backup_cat:
        backup_cat.write(source_cat.read())
```

Case: program logging 案例：程序日志

```
with open('app.log', 'a') as log_file:
    log_file.write('Program started\n')
    log_file.write('Operation completed\n')
```

WHEN TO USE `WITH` 何时使用 `with`

Always. For files, database connections and network resources alike. The manual `open`/`close` pair in this chapter exists so you understand what `with` is doing for you.

永远使用。文件、数据库连接、网络资源皆然。本章保留手工 `open`/`close`，是为了让你理解 `with` 替你做了什么。

KEY POINTS 本章要点

- Files are persistent containers; text is for humans, binary is for machines.

  文件是持久化容器；文本给人看，二进制给机器用。

- `r` reads, `w` overwrites instantly, `a` appends.

  `r` 读取，`w` 立即覆盖，`a` 追加。

- `read()`/`readline()`/`readlines()` differ in granularity; `seek(0)` rewinds.

  `read()`/`readline()`/`readlines()` 粒度不同；`seek(0)` 回拨。

- Reading consumes; `strip()` every line.

  读取即消耗；每行都要 `strip()`。

- `with` guarantees closure even on errors — default to it.

  `with` 保证即使出错也关闭——默认使用它。

EXERCISES 练习

- Write `classlist.txt`, then a script that greets every student on it, one per line.

  创建 `classlist.txt`，再写脚本逐行问候名单上的每位学生。

- Create `groceries.txt` with `write()` and `shopping.txt` with `writelines()`; print both to verify.

  分别用 `write()` 和 `writelines()` 创建 `groceries.txt` 与 `shopping.txt`，打印验证。

- Append two dated entries to `diary.txt` across two runs of the program, and show the file keeps all entries.

  分两次运行程序向 `diary.txt` 追加两条带日期的记录，证明文件保留了全部记录。

- Rewrite your class-list greeter with the `with` statement and explain in one sentence what changed.

  用 `with` 重写班级问候脚本，并用一句话说明改动了什么。

- Copy a binary file (any image) with the nested `with` pattern and compare file sizes.

  用嵌套 `with` 模式复制一个二进制文件（任意图片），并对比文件大小。

File Systems and Encodings / 文件系统与字符编码

> *A path tells you where; an encoding tells you what.*
>
> 路径回答在哪里；编码回答是什么。

LEARNING OBJECTIVES 学习目标

- Check existence and type of paths with `os.path`.

  用 `os.path` 检查路径的存在性与类型。

- Create, list and rename folders; measure file sizes.

  创建、列出、重命名文件夹；测量文件大小。

- Explain ASCII, UTF-8 and GBK, and convert between encodings.

  解释 ASCII、UTF-8 与 GBK，并进行编码转换。

- Open text files with explicit encodings and handle decode errors.

  以显式编码打开文本文件并处理解码错误。

13.1 The `os` Module

`os` 模块

Beyond reading and writing *content*, Python's `os` module lets your program manage the file system itself: checking, creating, listing, renaming.

除了读写*内容*，Python 的 `os` 模块还能让程序管理文件系统本身：检查、创建、列出、重命名。

Checking paths 检查路径

```
import os

print(os.path.exists("diary.txt"))   # True or False / 是否存在
print(os.path.isfile("photos/"))     # False — it is a folder / 是文件吗
print(os.path.isdir("photos/"))      # True / 是文件夹吗
```

Checking before opening prevents the classic `FileNotFoundError` — and lets you write programs that behave differently for existing and missing files.

打开前先检查，可以避免经典的 `FileNotFoundError`——还能让程序对"存在"与"缺失"做出不同反应。

Sizes and directories 大小与目录

```
size = os.path.getsize("secrets.txt")
print("File size: " + str(size) + " bytes")   # 1024 bytes = 1 KB / 1024 字节 = 1 KB

os.mkdir("experiments")        # create one folder / 建一个文件夹
# os.mkdir("projects/2024")    # ERROR: parent must exist first / 报错：父目录必须先存在
os.makedirs("projects/2024")   # creates the whole chain / 整条链路一次建好

files = os.listdir("photos/")  # names only, not full paths / 只有名字，没有完整路径
os.rename("old.txt", "new.txt")     # rename a file / 重命名文件
os.rename("temp/", "backup/")       # rename a folder / 重命名文件夹
```

13.2 Three Real-World Automations

三个现实世界的自动化

Auto-create a daily folder 自动创建每日文件夹

```
from datetime import datetime
import os

today = datetime.now().strftime("%Y%m%d")
os.mkdir("lab_" + today)
print("Created: lab_" + today)
```

Perfect for organizing lab results or daily logs — a new folder every day, named by the machine, never mistyped by a human.

非常适合整理实验结果或每日日志——每天一个新文件夹，由机器命名，绝不因人手输错。

Batch rename photos 批量重命名照片

Python string `replace()` plus a loop renames hundreds of files in seconds:

字符串 `replace()` 加一个循环，几秒钟重命名数百个文件：

```
for file in os.listdir("photos/"):
    if file.startswith("photo"):
        new_name = file.replace("photo", "vacation")
        os.rename("photos/" + file, "photos/" + new_name)
```

Find all text files 查找所有文本文件

```
text_files = []
for item in os.listdir("text/"):
    if item.endswith('.txt'):      # cleaner than slicing / 比切片更清晰
        text_files.append(item)

print("Found text files:", text_files)
```

13.3 Why Encoding Matters

为什么编码重要

Computers only understand 0s and 1s. An **encoding** is the translation dictionary between human-readable text and numeric binary — and using the wrong dictionary produces garbage.

计算机只认识 0 和 1。编码是"人类可读文本"与"数字二进制"之间的翻译字典——用错字典就会产生乱码。

ASCII ASCII

The founding alphabet: 128 characters — English letters, digits, basic symbols.

最初的字母表：128 个字符——英文字母、数字、基本符号。

```
print(ord('A'))   # 65 — the code behind the character / 字符背后的编码
print(chr(65))    # 'A' — the character behind the code / 编码背后的字符
```

UTF-8 UTF-8

The global standard: every language on Earth, using 1–4 bytes per character. It is Python 3's default encoding.

全球通行标准：覆盖地球上的所有语言，每个字符 1–4 字节。它是 Python 3 的默认编码。

```
print(len("中文".encode('utf-8')))   # 6 — 3 bytes per Chinese character / 每个汉字 3 字节
```

GBK GBK

A legacy standard for Simplified Chinese, fixed 2 bytes per Chinese character, common in older Windows systems.

简体中文的传统标准，每个汉字固定 2 字节，常见于较老的 Windows 系统。

```
print(len("中文".encode('gbk')))     # 4 — 2 bytes per character / 每个汉字 2 字节
```

Encoding	Coverage	Chinese character	Status
ASCII	English only 仅英文	impossible 无法表示	legacy 历史
UTF-8	All languages 全部语言	3 bytes 3 字节	modern default 现代默认
GBK	Simplified Chinese 简体中文	2 bytes 2 字节	legacy systems 旧系统

13.4 Encodings in Practice
编码实战

Always declare the encoding 永远显式声明编码

```
file = open('data.txt', 'r', encoding='utf-8')
content = file.read()
file.close()
```

Skipping the parameter works until the day your script meets a file saved on another machine — and then fails with `UnicodeDecodeError` (cannot decode bytes) or `UnicodeEncodeError` (cannot encode characters).

省略参数一直没问题，直到某天你的脚本遇到另一台机器上保存的文件——然后以 `UnicodeDecodeError`（无法解码字节）或 `UnicodeEncodeError`（无法编码字符）失败。

Tolerant modes exist for damaged files:

对损坏的文件可以使用宽容模式：

```
f1 = open('data.txt', 'r', encoding='utf-8', errors='ignore')    # skip bad chars / 跳过坏字符
f2 = open('data.txt', 'r', encoding='utf-8', errors='replace')   # replace with ? / 用 ? 替换
```

Case: converting GBK to UTF-8 案例：GBK 转 UTF-8

```
with open('old_data.txt', 'r', encoding='gbk') as source:
    content = source.read()

with open('new_data.txt', 'w', encoding='utf-8') as target:
    target.write(content)
```

Case: reading a Chinese CSV 案例：读取中文 CSV

The encoding must match the file's origin:

编码必须与文件来源匹配：

```
# data.csv contains: 姓名,年龄 / 张三,25 / 李四,30
with open('data.csv', 'r', encoding='gbk') as file:
    for line in file:
        name, age = line.strip().split(',')
        print("Name: " + name + ", Age: " + age)
```

UNKNOWN ENCODING? 编码未知?

The `chardet` library detects it for you: read the file in binary (`'rb'`) and call `chardet.detect(rawdata)['encoding']`. Install it with `pip install chardet`.

`chardet` 库可以代劳：以二进制（`'rb'`）读取文件，调用 `chardet.detect(rawdata)['encoding']`。安装命令：`pip install chardet`。

KEY POINTS 本章要点

- `os.path` answers "does it exist? is it a file?"; `os.mkdir`/`makedirs` create folders.

  `os.path` 回答"存在吗？是文件吗？"；`os.mkdir`/`makedirs` 创建文件夹。

- `os.listdir` gives names; `os.rename` renames files and folders.

  `os.listdir` 给出名字；`os.rename` 重命名文件与文件夹。

- ASCII = 128 English characters; UTF-8 = global, 3 bytes per Chinese char; GBK = 2 bytes.

  ASCII 为 128 个英文字符；UTF-8 全球通用、汉字 3 字节；GBK 汉字 2 字节。

- Always pass `encoding=` when opening text files.

  打开文本文件时永远传 `encoding=`。

- `errors='ignore'`/`'replace'` rescue damaged files; `chardet` detects encodings.

  `errors='ignore'`/`'replace'` 抢救损坏文件；`chardet` 检测编码。

EXERCISES 练习

- Write `path_check.py` that reports existence, type and byte size for three paths of your choice.

编写 `path_check.py`，对你选定的三个路径报告存在性、类型与字节大小。

- Write `daily_folder.py` that creates `lab_YYYYMMDD` for today and prints the result.

  编写 `daily_folder.py`，为今天创建 `lab_YYYYMMDD` 文件夹并打印结果。

- Write `batch_rename.py`: create `photos/photo_001.jpg` and `photo_002.jpg`, then rename both to `vacation_*`.

  编写 `batch_rename.py`：创建 `photos/photo_001.jpg` 与 `photo_002.jpg`，再重命名为 `vacation_*`。

- Create a GBK file, convert it to UTF-8 with Python, and open both in a text editor to compare.

  创建一个 GBK 文件，用 Python 转成 UTF-8，并在文本编辑器中打开对比。

- Write `csv_reader.py` that reads the Chinese CSV above with the correct encoding, then deliberately uses the wrong one and records the error.

  编写 `csv_reader.py`，用正确编码读取上面的中文 CSV；再故意用错编码并记录报错。

Errors and Exception Handling / 错误与异常处理

> *Errors are not failures of your program. They are messages from it.*
>
> 错误不是程序的失败，而是程序发来的消息。

LEARNING OBJECTIVES 学习目标

- Recognize the five most common built-in exceptions.
  识别五种最常见的内置异常。
- Wrap risky code in `try`/`except`, including multiple and tuple-style handlers.
  用 `try`/`except` 包裹风险代码，包括多分支与元组式处理。
- Use `else` and `finally` to build the complete pattern.
  使用 `else` 与 `finally` 构建完整模式。
- Raise exceptions deliberately with `raise`.
  用 `raise` 主动抛出异常。
- Apply exception handling to network requests and retries.
  将异常处理应用于网络请求与重试。

14.1 When Coding Meets Real Life

当编程遇上现实生活

Errors are an inevitable part of programming, like unexpected events in real life. Two classics:

错误是编程中不可避免的一部分，正如现实生活中的意外。两个经典场景：

```
open("non_existent_file.txt")
# like searching for a classroom that doesn't exist / 像在找一个根本不存在的教室
# FileNotFoundError

total_people = 0
cake_slices = 8 / total_people
# like preparing cake slices when nobody comes / 像在无人赴约时准备蛋糕
# ZeroDivisionError
```

The five you will meet most 最常遇到的五种

Error	Trigger
`SyntaxError`	`while True` — missing colon / 缺少冒号
`NameError`	`print(new_variable)` — never created / 使用了未创建的名字
`TypeError`	`1 + 'a'` — incompatible types / 不兼容的类型
`FileNotFoundError`	`open('okokokokok.txt', 'r')` — no such file / 文件不存在
`IndentationError`	a block with no 4-space indent / 代码块缺少 4 空格缩进

You practiced reading tracebacks in Chapter 2; this chapter is about *preventing the crash* — or at least making it graceful.

第 2 章你已经练习过阅读 traceback；本章讲的是*防止崩溃*——至少让崩溃变得体面。

14.2 `try` and `except`

`try` 与 `except`

Exception handling prevents your program from crashing when an error occurs. `try` holds the risky code; `except` is the seatbelt.

异常处理让程序在出错时不再崩溃。`try` 装着风险代码；`except` 是安全带。

```
try:
    result = 10 / 0            # risky code / 可能失败的代码
except ZeroDivisionError:
    print("Oops! Can't divide by zero!")   # the response / 应对方式
```

Case: the broken calculator 案例：搞坏计算器

```
try:
    weight = 70
    height = 0
    bmi = weight / height
except ZeroDivisionError:
    print("Alert! Did you enter height as zero?")
```

The specific error is caught and converted into feedback — instead of a crash.

特定错误被捕获并转化为反馈——而不是崩溃。

Case: the file detective 案例：文件侦探

```
try:
    with open("ghost_file.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("Pssst... this file doesn't exist!")
```

Friendly messages 友好提示

When building software for other people, convert technical errors into human language:

为他人开发软件时，要把技术错误翻译成人话：

```
try:
    user_age = int(input("Enter age: "))
except ValueError:
    print("Hey! Numbers only please :)")
```

14.3 Multiple Exceptions

多种异常

Stack several `except` blocks to respond differently to different failures. Python checks them top-to-bottom and runs **only the first matching** one.

堆叠多个 `except` 块以区别应对不同失败。Python 自上而下检查，**只执行第一个匹配的块**。

```
try:
    age = int(input("Age: "))
    print(str(100 / age))
except ValueError:
    print("Numbers only!")
except ZeroDivisionError:
    print("Age can't be zero!")
```

Input `0` triggers `ZeroDivisionError`; input `string` triggers `ValueError`; input `4` succeeds and prints `25.0`.

输入 `0` 触发 `ZeroDivisionError`；输入 `string` 触发 `ValueError`；输入 `4` 成功并打印 `25.0`。

Tuple-style catching 元组式捕获

If several errors deserve the *same* response, group them:

如果多个错误需要*同样*的应对，可以分组：

```
try:
    with open("grades.txt") as f:
        print(f.read())
except (FileNotFoundError, PermissionError):
    print("File problem! Check the file exists and you have access.")
```

But do not group everything blindly. In a calculator dividing by zero and typing letters deserve *different* advice — separate handlers give users tailored guidance.

但不要盲目全分组。计算器里"除以零"和"输入字母"需要*不同*的提示——分开处理才能给出针对性的指导。

The catch-all 兜底网

When you cannot predict what might go wrong, `Exception` catches everything and stores the details in `e`:

当无法预知会出什么错时，`Exception` 捕获一切并把细节存入 `e`：

```
try:
    magic_number = "100" + 5
except Exception as e:
    print(f"An error occurred: {e}")
    # can only concatenate str (not "int") to str
```

ORDERING MATTERS 顺序很重要

`Exception` includes `ValueError`, so `except (ValueError, Exception)` is redundant and hides information. Always list specific errors first and leave the catch-all as the last line of defense.

`Exception` 包含 `ValueError`，所以 `except (ValueError, Exception)` 是冗余的，还会掩盖信息。具体错误永远写在前面，兜底网只放在最后。

14.4 `else` and `finally`

`else` 与 `finally`

The complete pattern has four rooms, each with a precise job:

完整模式有四个房间，各司其职：

Block	Runs when
`try`	Always attempted first

	永远首先尝试
`except`	Only if the named error occurred 仅当发生对应错误
`else`	Only if the `try` block was 100% successful 仅当 `try` 完全成功
`finally`	Always — success, failure, or anything else 永远执行——无论成败

Why `else` exists `else` 存在的意义

Keeping success logic in `else` keeps the `try` block short and focused on what can actually fail. Without it, the success code sits inside `try` and *its* bugs get caught by the `except` — masking real bugs as handled errors.

把成功逻辑放进 `else`，可以让 `try` 保持短小、只聚焦真正可能失败的部分。否则成功代码也在 `try` 里，它自身的 bug 会被 `except` 捕获——把真正的 bug 伪装成"已处理的错误"。

A complete function 一个完整的函数

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Division by zero!")
    else:
        print("Result is " + str(result))
    finally:
        print("Operation complete")

safe_divide(10, 2)   # Result is 5.0 / Operation complete
safe_divide(5, 0)    # Division by zero! / Operation complete
```

A safe config loader 安全的配置加载器

```
config_file = False
try:
    config_file = open("game_config.cfg")
except FileNotFoundError:
    print("Config file missing!")
else:
    settings = config_file.read()
    print("Game configuration loaded!")
finally:
    if config_file:
        config_file.close()
    print("Resource cleanup completed")
```

Read it once more and narrate each room: open (may fail), complain (if missing), read (only if opened), close (always, but only if opened).

再读一遍并复述每个房间：打开（可能失败）、抱怨（缺失时）、读取（仅当打开成功）、关闭（永远执行，但仅在打开过时才有关可闭）。

14.5 `raise`: Errors on Purpose

`raise`：故意抛错

You can trigger an error yourself when a *business rule* is violated — not the machine's rules, but yours.

当被违反的不是机器的规则而是你的*业务规则*时，你可以亲手抛出错误。

```
existing_users = ["alice", "bob", "charlie"]

try:
    new_user = input("Choose username: ")
    if new_user in existing_users:
        raise ValueError("Username taken!")
    print("Username available!")
except ValueError as e:
    print(e)
```

Password strength 密码强度检查

```
def check_password(pwd):
    if len(pwd) < 8:
        raise Exception("Password too short!")
    if not any(c.isdigit() for c in pwd):
        raise Exception("Add numbers!")

try:
    check_password("weak")
except Exception as e:
    print("Weak password: " + str(e))
```

`raise` turns silent bad data into a loud, catchable event — the difference between a bug discovered by a user and a rule enforced by the program.

`raise` 把无声的坏数据变成响亮的、可捕获的事件——这就是"用户发现的 bug"与"程序执行的规则"之间的区别。

14.6 In the Wild: Networks and Retries

实战：网络与重试

Connection timeout 连接超时

```
import requests
try:
    response = requests.get("https://www.baidu.com", timeout=3)
except requests.exceptions.ConnectTimeout:
    print("Oops! Connection took too long!")
```

The program waits up to 3 seconds and then handles the timeout gracefully instead of hanging forever or crashing.

程序最多等待 3 秒，然后优雅地处理超时，而不是永远挂起或直接崩溃。

The retry loop 重试循环

```
import time, requests

for attempt in range(3):
    try:
        requests.get("https://www.baidu.com", timeout=3)
        print("Success!")
        break                       # exit on success / 成功即退出
    except Exception:
        wait_time = 3 - attempt     # 3s, then 2s, then 1s / 逐次递减
        print("Attempt " + str(attempt + 1) + " failed")
        time.sleep(wait_time)
```

The dynamic wait — 3, 2, 1 seconds — comes from arithmetic on the loop counter. This retry-then-escalate pattern appears everywhere in production systems.

动态等待 3、2、1 秒来自对循环计数器的算术。这种"重试再升级"的模式在生产系统中无处不在。

Input validation, end to end 端到端的输入验证

```
try:
    fahr = float(input("Enter Fahrenheit (-100 to 212F): "))
    if not (-100 <= fahr <= 212):
        raise ValueError("Unrealistic temperature!")
except ValueError as e:
    if "Unrealistic" in str(e):
        print("Invalid input: " + str(e))
    else:
        print("Numbers only please!")
else:
    celsius = (fahr - 32) * 5 / 9
    print(str(fahr) + "F = " + str(round(celsius, 1)) + "C")
```

Three defenses in eleven lines: type checking, range checking, and a graceful conversion path.

十一行代码包含三重防御：类型检查、范围检查，以及优雅的转换路径。

KEY POINTS 本章要点

- Know the big five: `SyntaxError`, `NameError`, `TypeError`, `FileNotFoundError`, `IndentationError`.

  熟记五巨头：`SyntaxError`、`NameError`、`TypeError`、`FileNotFoundError`、`IndentationError`。

- `try`/`except` converts crashes into feedback; specific handlers first, catch-all last.

  `try`/`except` 把崩溃变成反馈；具体处理器在前，兜底在后。

- `else` holds success logic; `finally` always runs cleanup.

  `else` 放成功逻辑；`finally` 永远负责清理。

- `raise` enforces your own business rules loudly.

  `raise` 让你自己的业务规则响亮生效。

- Timeouts and retry loops make network code production-grade.

  超时与重试循环让网络代码达到生产级。

EXERCISES 练习

- Write `bmi_calculator.py` wrapped in `try`/`except` handling both zero height and non-numeric input.

  编写 `bmi_calculator.py`，用 `try`/`except` 同时处理零身高与非数字输入。

- Write `file_detective.py` and `catch_all.py` from this chapter and record their outputs.

  按本章编写 `file_detective.py` 与 `catch_all.py`，并记录输出。

- Write `multiple_exceptions.py`; test inputs 0, `string` and 4, and explain each result.

  编写 `multiple_exceptions.py`；测试输入 0、`string`、4，并逐一解释结果。

- Rewrite the grade calculator in tuple style, then argue whether the tuple version is better.

  将成绩计算器改写为元组式捕获，再论证元组版本是否更优。

- Write `safe_config.py` and `safe_divide.py`; annotate each block with one line of comment.

编写 `safe_config.py` 与 `safe_divide.py`；为每个块写一行注释。

- Write `retry_dynamic.py` with decreasing wait times, then username checking with `raise` inside a `while` loop.

  编写等待时间递减的 `retry_dynamic.py`；再在 `while` 循环中用 `raise` 实现用户名查重。

Third-Party Libraries and pip / 第三方库与 pip

> *Great programmers stand on the shoulders of giants — and giants publish on PyPI.*
>
> 优秀的程序员站在巨人肩上——而巨人们都把成果发布在 PyPI 上。

LEARNING OBJECTIVES 学习目标

- Explain the hierarchy module, package, library.
  解释模块、包、库的层级关系。
- Install, upgrade, inspect and remove packages with pip.
  用 pip 安装、升级、查看、移除包。
- Configure a mirror for faster downloads in China.
  配置镜像源，加速国内下载。
- Fetch live data from the web with `requests`.
  用 `requests` 从网络获取实时数据。

15.1 Module, Package, Library

模块、包、库

Three words, three sizes. Keeping them straight makes documentation readable.

三个词，三种尺寸。分清它们，文档就好读了。

Term	Definition
Module 模块	A single `.py` file with functions, classes and variables; the basic unit of organization 单个 `.py` 文件，含函数、类与变量；代码组织的基本单位
Package 包	A folder containing multiple modules 包含多个模块的文件夹
Library 库	A functional collection that may span many packages and modules (e.g. machine learning, web scraping) 跨多个包与模块的功能集合（如机器学习、网页抓取）

15.2 pip: The Package Manager

pip：包管理器

Third-party libraries live on **PyPI.org** — the Python Package Index — and `pip` downloads them for you. Run pip from the *command line*, not inside the Python interpreter.

第三方库托管在 **PyPI.org**（Python 包索引）上，`pip` 替你下载它们。请在*命令行*运行 pip，而不是在 Python 解释器里。

```
pip3 --version                          # check the version / 检查版本
pip3 install requests                   # install a package / 安装包
pip3 install requests==2.31.0           # install a specific version / 安装指定版本
pip3 install --upgrade requests         # upgrade / 升级
pip3 list                               # everything installed / 已安装清单
pip3 show requests                      # details of one package / 单个包详情
pip3 freeze                             # dependencies in requirements format / 依赖清单格式
```

Mirrors 镜像源

Downloading from the official PyPI can be slow from China. Local mirrors — Tsinghua University, AliCloud, USTC — host the same packages much closer:

从官方 PyPI 下载在国内可能很慢。本地镜像——清华大学、阿里云、中科大——在更近的位置提供相同的包：

```
# temporary, one install / 临时，单次安装
pip3 install -i https://pypi.tuna.tsinghua.edu.cn/simple pandas

# permanent configuration / 永久配置
pip3 config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

15.3 `requests`: Talking to the Web

`requests`：与网络对话

`requests` is the de-facto HTTP library of Python. After `pip3 install requests`:

`requests` 是 Python 事实上的 HTTP 库。执行 `pip3 install requests` 之后：

```
import requests

response = requests.get("https://www.baidu.com")
```

```
print("Status Code: " + str(response.status_code))   # 200 means OK / 200 表示成功
print("Response Text: " + response.text[:100] + "...")
```

A status code of `200` means success; codes in the 400s mean you asked for something unavailable; the 500s mean the server stumbled. Reading status codes is the first step of every web debugging session.

状态码 `200` 表示成功；4 开头表示你请求的东西不存在；5 开头表示服务器出错了。读懂状态码是每次网页调试的第一步。

15.4 Project: The Weather Reporter

项目：天气预报员

Combine web fetching (`requests`) and time handling (`datetime`) into a genuinely useful tool.

把网络抓取（`requests`）与时间处理（`datetime`）组合成一个真正有用的工具。

```
import datetime
import requests

def get_weather(city):
    # %t = temperature, %C = condition / %t 温度，%C 天气状况
    url = "http://wttr.in/" + city + "?format=%t+%C"
    response = requests.get(url)
    return response.text

def weather_report(city):
    time_now = datetime.datetime.now().strftime("%H:%M")
    weather = get_weather(city)
    return "[" + time_now + "] Weather in " + city + ": " + weather

print(weather_report("London"))
print(weather_report("NewYork"))
```

THINK ABOUT IT 想一想

Something is subtly wrong with this program. Hint: we are in Beijing.

这个程序有个不易察觉的问题。提示：我们在北京。

`datetime.datetime.now()` returns the *local time of the machine running the script* — the server, not the city being queried. A report about New York shows Beijing time. Fixing it properly requires timezone handling (the `pytz` library or `datetime.timezone`); the first step is simply noticing.

`datetime.datetime.now()` 返回的是*运行脚本的机器的本地时间*——是服务器的时间，而不是被查询城市的时间。关于纽约的天气预报显示的是北京时间。彻底修复需要时区处理（`pytz` 库

> 或 `datetime.timezone`）；第一步是先察觉到这个问题。

15.5 Project: The Backup Helper

项目：文件备份助手

`os` and `shutil` together automate file management:

`os` 与 `shutil` 联手实现文件管理自动化：

```
import os
import shutil

def backup_files(src_folder, dst_folder):
    # 1. create the destination if missing / 目标不存在则创建
    if not os.path.exists(dst_folder):
        os.makedirs(dst_folder)
        print("Created backup directory: " + dst_folder)

    # 2. copy every .txt file / 复制每个 .txt 文件
    for file in os.listdir(src_folder):
        if file.endswith(".txt"):
            src_path = os.path.join(src_folder, file)
            dst_path = os.path.join(dst_folder, file)
            shutil.copy2(src_path, dst_path)
            print("Copied " + file + "!")

backup_files("notes", "backup")
```

`os.path.join()` builds paths correctly on every operating system — never glue strings with `+` yourself.

`os.path.join()` 在任何操作系统上都能正确拼路径——永远别用 `+` 自己拼。

Function	Difference
`shutil.copy()`	Copies content and permissions 复制内容与权限
`shutil.copy2()`	Also copies metadata (timestamps) — best for backups 还复制元数据（时间戳）——备份首选

`shutil.copytree()` copies an entire folder tree in one call — the one-liner behind most backup scripts.

`shutil.copytree()` 一次调用复制整个文件夹树——大多数备份脚本的核心就这一行。

KEY POINTS 本章要点

- Module = file, package = folder, library = functional collection.

  模块 = 文件，包 = 文件夹，库 = 功能集合。

- pip installs from PyPI; configure a Tsinghua/AliCloud mirror in China.

  pip 从 PyPI 安装；国内配置清华/阿里云镜像。

- `requests.get()` plus status codes is the start of every web integration.

  `requests.get()` 加状态码是一切网络集成的起点。

- `os.path.join` for paths; `shutil.copy2` for backups; `copytree` for folders.

  路径用 `os.path.join`；备份用 `shutil.copy2`；整树复制用 `copytree`。

- `datetime.now()` is machine-local time — be alert to time zones.

  `datetime.now()` 是机器本地时间——警惕时区问题。

EXERCISES 练习

- Run the full pip sequence in Section 15.2 on the server and screenshot `pip3 list`.

  在服务器上运行 15.2 节的完整 pip 流程，并截图 `pip3 list`。

- Write `requests_demo.py` fetching baidu.com, printing the status code and the first 100 characters.

  编写 `requests_demo.py` 抓取 baidu.com，打印状态码与前 100 个字符。

- Write `weather_reporter.py` and report for two cities; identify the timezone problem in your report.

  编写 `weather_reporter.py` 查询两个城市；在报告中指出时区问题。

- Write `backup_helper.py`: create `notes/` with two `.txt` files, run the backup, then verify destination contents.

  编写 `backup_helper.py`：在 `notes/` 放两个 `.txt`，运行备份，并核对目标内容。

Doing Several Things at Once / 多任务处理

> *Waiting is the most common thing a program does. Concurrency is how you stop wasting it.*
>
> 等待是程序做得最多的事；并发就是不再浪费它。

LEARNING OBJECTIVES 学习目标

- Classify tasks as I/O-bound or CPU-bound.
  把任务分为 I/O 密集与 CPU 密集。
- Choose between threading, subprocess and multiprocessing.
  在多线程、子进程、多进程之间做选择。
- Start threads, pass them arguments, and collect results via a queue.
  启动线程、传参，并通过队列收集结果。
- Explain blocking versus non-blocking execution.
  解释阻塞与非阻塞执行。

16.1 What Is Multi-tasking?

什么是多任务处理?

Multi-tasking lets a program perform several operations *concurrently* — seemingly at the same time. It pays off when tasks spend their lives waiting (for the network, for the disk) or when heavy computation can be split.

多任务处理让程序*并发地*执行多个操作——看起来同时进行。当任务一生都在等待（等网络、等磁盘），或繁重计算可以拆分时，并发才开始产生收益。

Python offers three main approaches:

Python 提供三种主要方式：

Approach	Best for	Examples
Threading	I/O-bound tasks	Web requests, file downloads

多线程	I/O 密集任务	网络请求、文件下载
Subprocess 子进程	Running external programs 运行外部程序	Calling ffmpeg, system commands 调用 ffmpeg、系统命令
Multiprocessing 多进程	CPU-bound tasks CPU 密集任务	Heavy computation, data processing 繁重计算、数据处理

16.2 Blocking and Non-blocking

阻塞与非阻塞

Two words describe how a program waits:

两个词描述程序如何等待：

- **Blocking**: the program stops and waits until a resource is available. Nothing else happens.
  阻塞：程序停下等待资源，期间什么都不会发生。
- **Non-blocking**: the program starts a task in the background and keeps working.
  非阻塞：程序把任务放到后台，自己继续干活。

A five-second `time.sleep()` in the main flow blocks everything. In a thread, the same sleep runs while the main program carries on.

主流程里一个 5 秒的 `time.sleep()` 会阻塞一切。放进线程里，同样的睡眠期间主程序照常工作。

16.3 Threading

多线程

The basic pattern 基本模式

```
from threading import Thread
import time

def blocking_task():
    print("Background task started...")
    time.sleep(5)
    print("Background task finished!")

this_thread = Thread(target=blocking_task)
this_thread.start()                      # runs in the background / 后台运行
```

```
print("Main thread continues working immediately!")   # 主线程立即继续

this_thread.join()                          # wait for it to finish / 等它完成
print("Is thread alive? " + str(this_thread.is_alive()))   # False
```

Three verbs run the show: `start()` launches the thread, `is_alive()` asks about it, `join()` waits for it. Without `join()`, the main program may exit before the background task completes.

三个动词撑起一切：`start()` 启动线程，`is_alive()` 询问状态，`join()` 等待完成。没有 `join()`，主程序可能在后台任务完成前就退出了。

Passing arguments 传参

Arguments go through the `args` tuple — note the comma in `(15,)` that makes it a tuple:

实参通过 `args` 元组传入——注意 `(15,)` 里的逗号，它造出元组：

```
def sleep_task(sleep_time=5):
    print("Sleep time is: " + str(sleep_time))
    time.sleep(sleep_time)

this_thread = Thread(target=sleep_task, args=(3,))
this_thread.start()
this_thread.join()
```

Getting results back 取回结果

Threads share memory, but a function running in a thread cannot simply `return` to the caller. The standard solution is a thread-safe **queue**:

线程共享内存，但线程里跑的函数无法直接 `return` 给调用者。标准解法是线程安全的队列：

```
from threading import Thread
import queue
import time

def calculate_task(q, value):
    print("Calculating...")
    time.sleep(2)
    q.put(value * 2)          # put the result into the queue / 结果放进队列

q = queue.Queue()
this_thread = Thread(target=calculate_task, args=(q, 5))
this_thread.start()
this_thread.join()
print("Final result: " + str(q.get()))    # 10
```

16.4 Threading versus Subprocess

多线程与子进程

Feature	Threading	Subprocess
Memory 内存	Shared by all threads 所有线程共享	Isolated per process 每个进程隔离
Startup 启动	Fast 快	Slower 较慢
Use case 场景	I/O-bound tasks I/O 密集任务	External programs 外部程序
Communication 通信	Direct, but race-condition risk 直接，但有竞争风险	Explicit: pipes, stdout, files 显式：管道、标准输出、文件

Shared memory makes threads easy to communicate with — and easy to corrupt. Two threads appending to one list can interleave in surprising ways; this is the famous *race condition*. Processes never share memory, so they must talk through explicit channels, which is slower but safer.

共享内存让线程通信容易——也容易搞坏。两个线程往同一个列表里追加数据，交错方式可能出人意料；这就是著名的*竞争条件*。进程从不共享内存，只能通过显式通道交流，较慢但更安全。

SELF-STUDY CHALLENGE 自学挑战

Research the `subprocess` library in Python 3. How would you run a terminal command such as `ls -l` from a Python script and capture its output? Try it, and write five lines on when you would prefer subprocess over threading.

研究 Python 3 的 `subprocess` 库。如何在 Python 脚本中运行 `ls -l` 这样的终端命令并捕获输出？动手试试，并用五行文字说明何时子进程优于多线程。

KEY POINTS 本章要点

- Concurrency pays off for waiting (I/O) and for splitting heavy computation.
  并发的收益来自等待（I/O）与拆分繁重计算。

- Threading for I/O, subprocess for external programs, multiprocessing for CPU.

  I/O 用多线程，外部程序用子进程，CPU 用多进程。

- `start()`, `is_alive()`, `join()` control a thread's life.

  `start()`、`is_alive()`、`join()` 掌控线程生命周期。

- `args=(x,)` passes arguments; `queue.Queue()` carries results back.

  `args=(x,)` 传参；`queue.Queue()` 送回结果。

- Shared memory is powerful and dangerous: beware race conditions.

  共享内存强大而危险：警惕竞争条件。

EXERCISES 练习

- Write `thread_demo.py` with a 5-second background task; print "main continues" before it finishes.

  编写 `thread_demo.py`：后台任务 5 秒，在其结束前打印"主线程继续"。

- Write `thread_args.py` passing different sleep times to two threads.

  编写 `thread_args.py`，向两个线程传入不同的睡眠时长。

- Write `thread_queue.py`: two threads compute squares of different numbers; the main thread collects both results.

  编写 `thread_queue.py`：两个线程分别计算不同数字的平方，主线程收集两个结果。

- Compare: run the same three downloads sequentially and in three threads, and time both versions.

  对比：用顺序与三线程两种方式下载同样三个任务，并分别计时。

PART VI

The Final Project / 期末项目

Everything so far was training with weights on. The final project takes them off: you will build a Python program that talks to a large language model, generates a coherent movie script clip by clip, and turns it into a one-minute film. Functions, loops, strings, files, exceptions and libraries all meet here — on a deadline.

前面的一切都是负重训练。期末项目卸下负重：你将编写一个与大型语言模型对话的 Python 程序，逐段生成连贯的电影脚本，并把它变成一分钟短片。函数、循环、字符串、文件、异常、库在此汇聚——而且有截止日期。

The Python Movie Scriptwriter / Python 电影编剧程序

> *You are not generating a movie. You are generating the machine that generates the movie.*
>
> 你要生成的不是电影，而是那台生成电影的机器。

LEARNING OBJECTIVES 学习目标

- Read, understand and modify an existing multi-function program.
  阅读、理解并修改一个现成的多函数程序。
- Drive a large language model through its API with structured prompts.
  通过 API 用结构化提示词驱动大语言模型。
- Generate coherent content iteratively with loops.
  用循环迭代生成连贯内容。
- Package code into functions and deliver a live demonstration.
  将代码封装成函数并完成现场演示。

17.1 The Goal

项目目标

You will create a **Python Movie-Scriptwriter program** that automatically generates coherent prompts — movie scripts — for six 10-second movie clips, producing a one-minute mini-movie.

你将创建一个 **Python 电影编剧程序**：自动为六段 10 秒的电影片段生成连贯的提示词（电影脚本），最终产出一分钟连贯短片。

Keywords of this project: Python, LLM, API, Prompt, Movie.

本项目关键词：Python、LLM、API、Prompt、电影。

Component	Tool
AI assistant on the server	Based on the Kimi API, at `/home/toolkit/tools/ai/ai.py`

服务器上的 AI 助手	基于 Kimi API，位于 `/home/toolkit/tools/ai/ai.py`
Image and video generation 图像与视频生成	Local ComfyUI, or online DouBao / JiMeng 本地 ComfyUI，或在线豆包/即梦
Editing and stitching 剪辑与拼接	JianYing (剪映) 剪映

17.2 Study the Reference Program

研读参考程序

A working reference implementation is provided. Your first task is not to write code — it is to **read** code: run it, trace it, and explain it.

项目提供了可运行的参考实现。你的第一个任务不是写代码，而是读代码：运行它、追踪它、解释它。

The reference program is organized as a small pipeline of functions, each wrapping one conversation with the AI assistant:

参考程序被组织成一条由若干函数构成的小流水线，每个函数封装一次与 AI 助手的对话：

Function	Role
`buildOpen(start_content)`	From the film's requirements, generate an opening image prompt, then a 5-second clip prompt 根据影片整体要求生成开场画面提示词，再生成 5 秒片段提示词
`buildQuestion(start_content)`	Build a 20-second story framework that ends on a cliffhanger, then produce its clips iteratively 生成一个结尾留悬念的 20 秒故事框架，再迭代产出各片段
`buildValue(value_content)`	The same pattern applied to a 15-second advertisement 同样的模式应用于 15 秒广告
`buildAnwser(question_content)`	A sequel function that *resolves* the cliffhanger of `buildQuestion` 续写函数，消除 `buildQuestion` 留下的悬念
`run()`	The conductor: calls the builders in order 指挥：按顺序调用各构建函数

Inside these functions you will find every idea from this book at work:

这些函数内部，本书的每个思想都在场：

- **String operations**: prompts are assembled with concatenation from the user's idea, the framework so far, and the previous clip prompts.

  字符串操作：提示词由用户想法、已有框架和此前片段提示词拼接而成。

- **Loops**: each builder uses a `while` loop; every new clip prompt is generated *from the previous ones*, so coherence is iterative, not accidental.

  循环：每个构建函数都用 `while` 循环；每个新片段提示词都*基于之前的片段*生成——连贯性是迭代的产物，而非偶然。

- **Functions**: each builder is a named, reusable block; the whole program reads as a pipeline.

  函数：每个构建函数都是命名的可复用块；整个程序读起来像一条流水线。

- **Lists**: `start_video_prompts_list` accumulates clip prompts across loop iterations.

  列表：`start_video_prompts_list` 在循环中不断累积片段提示词。

> **THINK ABOUT IT** 想一想
>
> Why does each prompt include *all previous prompts* instead of only the last one? What would the movie look like if each clip only saw its immediate predecessor?
>
> 为什么每个提示词要包含*全部此前提示词*，而不只是上一个？如果每段只看到自己的前一段，电影会变成什么样？
>
> Each new prompt carries the framework plus the full list of existing clips, so the model keeps the whole story in view — like a scriptwriter re-reading the script before writing the next scene. With only the last clip in view, the story drifts: characters vanish, moods reset. Long-range coherence requires long-range memory.
>
> 每个新提示词都携带故事框架和已有的全部片段，让模型始终看到整个故事——就像编剧在写下一场戏之前重读剧本。只看上一段的话，故事会漂移：角色消失，情绪重置。长程连贯需要长程记忆。

17.3 Requirements

项目要求

Your submission must satisfy all of the following:

你的提交必须满足以下全部要求：

1. Your program must run on Linux.

   程序必须能在 Linux 上运行。

2. Your code must be modified from the reference at `/home/toolkit/tools/ai/ai.py` — not copied wholesale.

   代码必须基于 `/home/toolkit/tools/ai/ai.py` 修改——不得整体照抄。

3. The six clip prompts must be generated **iteratively**: each clip's prompt is based on the previous one, implemented with Python loop syntax.

   六个片段的提示词必须迭代生成：每个片段基于前一个，并使用 Python 循环语法实现。

4. Useful code blocks must be packed into functions.

   有用的代码块必须封装为函数。

5. User input is limited to a simple description of **under 20 English words**.

   用户输入限制为 **20 个英文单词以内**的简单描述。

17.4 Making the Mini-Movie

制作迷你电影

The program outputs prompts; you turn them into pixels:

程序输出提示词，你把提示词变成画面：

1. Generate each 5–10 second clip from its prompt, using local ComfyUI or the online DouBao / JiMeng services.

   用本地 ComfyUI 或在线豆包/即梦，按每段提示词生成 5–10 秒片段。

2. Import all clips into JianYing (剪映), arrange them in order, add sound if you like.

   将所有片段导入剪映，按顺序排列，可自行配乐。

3. Export a one-minute mini-movie.

   导出一分钟成片。

Document every step with screenshots as you go. The making-of material is part of your grade: your presentation must show the efforts behind the film, not only the film.

制作过程中逐步截图记录。制作花絮是评分的一部分：演示要展示电影背后的工作，而不只是电影本身。

17.5 Teams

团队

The class forms five teams of five to seven members, through multiple rounds of selection:

全班组成五支队伍，每队五至七人，经过多轮选择：

1. In each round, every Team Lead introduces themselves.
   每一轮中，各队长自我介绍。
2. Each Team Lead extends one offer to one or more members; you may reject an offer.
   每位队长向一名或多名成员发出邀请；你可以拒绝。
3. The final project maximum is 30 points (final presentation); the Team Lead earns 3 bonus points (10%).
   期末项目满分 30 分（现场展示）；队长获得 3 分加分（10%）。
4. If more than five students want to lead, we vote; if fewer, the remaining leads are assigned randomly.
   想当队长的人超过五个则投票；不足则随机指派。

Choose leads for organization, not just code: coordinating six people, six models and one deadline is the harder skill.

选队长看组织力，而不只是代码：协调六个人、六个模型和一个截止日期，才是更难的本事。

17.6 The Final Presentation

期末展示

30 minutes of presentation plus 5 minutes of questions:

30 分钟展示，5 分钟问答：

1. **Play the demo**: your one-minute mini-movie, with its six coherent clips.

   播放样片：由六个连贯片段组成的一分钟迷你电影。

2. **Live demo**: run your program on stage; generate six coherent movie scripts from an input of under 20 words.

   现场演示：在台上运行程序，从 20 词以内的输入生成六个连贯的电影脚本。

3. **Code walkthrough**: explain the implementation in detail, focusing on how functions and loop syntax produce the iterative prompts.

   代码讲解：详细解释实现，重点是函数与循环语法如何产生迭代式提示词。

4. **Making-of**: show all the work behind the mini-movie — failed generations included.

   制作花絮：展示迷你电影背后的全部工作——包括失败的生成。

Rehearse the live demo. Presentations fail on network hiccups and untested inputs, not on missing slides. Prepare a pre-generated fallback output in case the server is slow, and pick your on-stage input word in advance.

排练现场演示。演示失败往往因为网络抖动和未测试的输入，而不是缺幻灯片。准备一份预生成的备用输出以防服务器变慢，并提前选定台上使用的输入词。

KEY POINTS 本章要点

- Read the reference program before writing anything; explain every function in one sentence.

  动笔之前先读懂参考程序；每个函数都用一句话解释。

- Coherence is engineered: new prompts are built from the framework plus all previous clips, inside a loop.

  连贯是工程出来的：在循环中，用框架加全部已有片段构造新提示词。

- Five hard requirements: Linux, modification, iterative loops, functions, input under 20 words.

  五条硬要求：Linux、基于参考修改、迭代循环、函数封装、输入不超过 20 词。

- The deliverable is a film *and* an honest record of making it.

  交付物是一部电影，*以及*诚实的制作记录。

- Presentations are rehearsed, with fallbacks ready.

  演示必须排练，并备好兜底方案。

PART VII

Reference / 附录与参考资料

Linux Command Quick Reference / Linux 命令速查

The twenty commands of Chapter 1, in one place. Twenty commands, printed once, worth taping above your desk.

第 1 章的二十条命令汇总于此。就这二十条，打印一页，贴在桌前。

Command	Mnemonic	Action
`pwd`	print working directory 打印工作目录	Show the absolute path of where you are 显示当前位置的绝对路径
`ls`	list 列出	List files and folders here 列出此处的文件与文件夹
`cd <path>`	change directory 切换目录	Move to `<path>` 移动到 `<path>`
`cd ..`	dot-dot 两点	Move up one level 向上一级
`cd ~`	tilde 波浪号	Go to your home directory 回到主目录
`mkdir <name>`	make directory 建目录	Create a new folder 创建新文件夹
`touch <file>`	touch 触碰	Create an empty file 创建空文件
`cat <file>`	concatenate 连接打印	Print the entire file at once 一次性打印整个文件
`less <file>`	less is more 少即是多	View page by page; `q` quits 分页查看；`q` 退出
`cp <src> <dst>`	copy 复制	Copy a file 复制文件
`cp -r <s> <d>`	recursive 递归	Copy a folder with everything inside 复制文件夹及其全部内容
`mv <src> <dst>`	move	Move or rename

	移动	移动或重命名
`rm <file>`	remove 删除	Delete a file — permanently 永久删除文件
`rm -r <dir>`	recursive 递归	Delete a folder — permanently 永久删除文件夹
`python3 <file>`	run Python 运行 Python	Execute a script 执行脚本
`pip3 install <pkg>`	install 安装	Download a package from PyPI 从 PyPI 下载安装包
`pip3 list`	list 列出	Show installed packages 显示已安装的包
`nano <file>`	editor 编辑器	Open the nano editor 打开 nano 编辑器
`history`	history 历史	List all previous commands 列出此前所有命令
`Tab`	completion 补全	Auto-complete names; twice lists options 自动补全；连按两次列出候选

Python Reserved Keywords / Python 保留关键字

The 35 words that belong to the language (Python 3.7). None of them can be a variable or function name.

属于语言本身的 35 个单词（Python 3.7）。它们都不能用作变量名或函数名。

Keyword				
False	None	True	and	as
assert	async	await	break	class
continue	def	del	elif	else
except	finally	for	from	global
if	import	in	is	lambda
nonlocal	not	or	pass	raise
return	try	while	with	yield

A quick orientation for the ones this book used heavily: `def` defines functions, `if`/`elif`/`else` decide, `while`/`for` repeat, `break`/`continue` steer loops, `return` hands back values, `import`/`from` load modules, `in` tests membership, `and`/`or`/`not` combine conditions, `try`/`except`/`finally`/`raise` handle errors, `with` manages resources, `lambda` writes tiny inline functions.

本书高频使用者的快速定位：`def` 定义函数；`if`/`elif`/`else` 判断；`while`/`for` 循环；`break`/`continue` 控制循环；`return` 返回值；`import`/`from` 导入模块；`in` 测试成员；`and`/`or`/`not` 组合条件；`try`/`except`/`finally`/`raise` 处理异常；`with` 管理资源；`lambda` 写小型内联函数。

A Field Guide to Error Messages / 错误信息速查

Read every traceback bottom-up: the last line names the error, the lines above locate it.

读 traceback 一律自下而上：最后一行给出错误名，上面的行给出位置。

Error	Typical cause	First aid
`SyntaxError`	Missing colon, unbalanced quotes 缺冒号、引号不配对	Look at the named line and the one before 检查报错行及其上一行
`IndentationError`	Mixed tabs/spaces, wrong level Tab 与空格混用、层级错误	Re-indent to exactly 4 spaces 统一为 4 空格
`NameError`	Typo, or use before definition 拼写错误或先用后定义	Check spelling and order 检查拼写与定义顺序
`TypeError`	`str + int`, wrong argument types `str + int`、实参类型错误	Convert with `str()`/`int()` 用 `str()`/`int()` 转换
`ValueError`	`int("abc")` — right type, bad value 类型对但值无效	Validate input with `try` 用 `try` 校验输入
`IndexError`	Position beyond the sequence 越界访问	Check `len()` and the last valid index 检查 `len()` 与最大合法下标
`KeyError`	Dictionary key missing 字典键不存在	Use `.get(key, fallback)` 使用 `.get(key, fallback)`
`FileNotFoundError`	Wrong path or name 路径或名字错误	`ls` the folder; check cwd 用 `ls` 检查目录与当前路径
`ZeroDivisionError`	Dividing by zero 除以零	Guard the divisor with `if` 用 `if` 保护除数
`ModuleNotFoundError`	Module not installed / wrong folder 未安装或路径不对	`pip3 install`, or run from the right directory 安装模块，或在正确目录运行
`UnboundLocalError`	Assigning to a global without `global` 未加 `global` 就给全局变量赋值	Pass it in, or declare `global` 传参或声明 `global`

`UnicodeDecodeError`	Wrong encoding when reading 读取时编码用错	Reopen with `encoding='utf-8'` (or `'gbk'`) 用正确编码重新打开

Course Map and Assessment / 课程安排与考核

Weekly plan 周计划

Week	Content
w01	Linux basics Linux 基础
w02	Python basics Python 基础
w03	Phase Test I + SSS (2h) 阶段测验一 + 自习（2 小时）
w04	Control flow 控制流
w05	Functions and modules 函数与模块
w06	Phase Test II + SSS (2h) 阶段测验二 + 自习（2 小时）
w07	Data structures 数据结构
w08	File operations 文件操作
w09	SSS (4h) 自习（4 小时）
w10	Phase Test III + SSS (2h) 阶段测验三 + 自习（2 小时）
w11	Exception handling 异常处理
w12	Libraries and multi-tasking 库与多任务处理
w13	Phase Test IV + SSS (2h) 阶段测验四 + 自习（2 小时）

w14	Presentation guidance + SSS (3h) 展示指导 + 自习（3 小时）
w15	Final presentation 期末展示

Lecture/Seminar + Lab + Phase Tests + Presentation total 45 hours; Supervised Self Study (SSS) adds 15 hours.

课堂/研讨 + 实验 + 阶段测验 + 展示共 45 学时；督导自习（SSS）另加 15 学时。

Grading 成绩评定

The final grade (0 to 100) uses a quantile-based method across three components: general performance (attendance and weekly reports), the final project (presentation and slides), and the total test score (best 3 of 4 phase tests). For each component, a student's quantile is computed within the cohort (0 = lowest, 1.0 = highest), and the Average Quantile (AQ) is taken across the three.

最终成绩（0 到 100）采用分位数法，综合三个部分：平时表现（出勤与周报）、期末项目（展示与幻灯片）、测验总分（四次阶段测验取最高三次）。每个部分在全体范围内计算分位数（0 = 最低，1.0 = 最高），再取三者的平均分位数（AQ）。

```
if AQ >= 0.2:
    Passed. Final grade = (AQ - 0.2) / (AQmax - 0.2) * 40 + 60
elif Total Test Score >= passing line:
    Passed with a low final grade: 60
else:
    Failed. Final grade = AQ / 0.2 * 60
```

`AQmax` is the highest AQ actually achieved in the cohort. Note the structure: quantiles are relative, so your standing among classmates — not raw points — drives the grade.

`AQmax` 是本班级实际出现的最高 AQ。注意其结构：分位数是相对的，决定成绩的是你在班级中的位置——而不是绝对分数。

Rules 规则

- **Academic integrity**: no AI and no cheating in phase tests. Outside phase tests, make full use of LLMs and AI tools — that is professional practice.

学术诚信：阶段测验中禁止使用 AI、禁止作弊。测验之外，请充分利用大模型与 AI 工具——这是职业素养。

- **Retake policy**: there is no resit exam and no free retake. Students who fail must retake the whole course, assessed by phase tests and multiple tasks, under the same quantile rules, registering with the Academic Affairs Office by the stated deadline.

  重修政策：没有补考，没有免费重考。不及格者必须重修整门课程，考核方式为阶段测验加多项任务，沿用同样的分位数规则，并须在教务处规定期限内注册。

- **Weekly reports**: document every practice step with screenshots in a PPT, export to PDF, and submit to the Weekly Report Assignment on time — it is a major part of your general performance.

  周报：将每次练习的每一步截图整理进 PPT，导出 PDF，按时提交至"Weekly Report Assignment"——这是平时成绩的重要组成。

- **Office hour**: Wednesday 12:00–20:00, Teaching Building B526, by email appointment at least one day ahead (feng.zhang@hibiuh.edu.cn).

  答疑时间：每周三 12:00–20:00，教学楼 B526，请至少提前一天邮件预约（feng.zhang@hibiuh.edu.cn）。

Glossary / 术语表

Term	Meaning
absolute path 绝对路径	Full address from the root to a file 从根目录到文件的完整地址
API 应用程序接口	Contract through which programs talk to each other, e.g. to an LLM 程序之间对话的约定，如与大语言模型交互
argument 实参	The actual value passed to a function at call time 调用时传给函数的真实值
binding 绑定	Linking a name to an object in memory 把名字与内存中的对象关联
boolean 布尔型	Data type with values True and False 取值为 True 与 False 的数据类型
CLI 命令行界面	Text-based interface; the terminal is its typical form 基于文本的界面；终端是其典型形态
dictionary 字典	Key-value container with fast lookup 以键值对组织、查找极快的容器
dynamic typing 动态类型	Types determined at run time; a name may change type 类型在运行时确定，名字可更换类型
encoding 编码	Translation dictionary between text and bytes, e.g. UTF-8, GBK 文本与字节之间的翻译字典，如 UTF-8、GBK
exception 异常	An error object raised during execution, catchable with `try` 执行中抛出的错误对象，可用 `try` 捕获
f-string f 字符串	String with inline expressions: `f"{x}"` 支持内联表达式的字符串
function 函数	Named, reusable block of code, optionally taking parameters and returning values 有名字、可复用的代码块，可带参数并返回值
garbage collection 垃圾回收	Automatic reclaiming of data no name points to 自动回收没有任何名字指向的数据
GUI	Interface of windows, icons and pointers

图形用户界面	由窗口、图标、指针构成的界面
immutable 不可变	Cannot be changed after creation, e.g. tuples, strings 创建后不可更改，如元组、字符串
index 索引	Position of an element in a sequence, starting at 0 元素在序列中的位置，从 0 开始
I/O-bound I/O 密集	Spending most time waiting for input/output 大部分时间用于等待输入输出
iterable 可迭代对象	Anything a `for` loop can visit item by item 能被 `for` 逐项访问的对象
keyword argument 关键字参数	Argument passed by name: `f(size=30)` 以名字传递的实参
list 列表	Ordered, mutable sequence 有序、可变的序列
local / global scope 局部/全局作用域	Where a name is visible: inside a function, or everywhere 名字可见的范围：函数内，或全局
loop 循环	Repetition of a block: `for`, `while` 代码块的重复执行
module 模块	A single `.py` file of reusable code 单个可复用代码的 `.py` 文件
mutable 可变	Can be modified in place, e.g. lists, dicts, sets 可原地修改，如列表、字典、集合
package 包	A folder of modules 由多个模块组成的文件夹
parameter 参数	The placeholder in a function definition 函数定义中的占位符
PEP8 Python 增强提案 8	The community style guide for Python code Python 社区的代码风格指南
pip 包管理器	The tool that installs packages from PyPI 从 PyPI 安装包的工具
prompt 提示词	Text instructing a language model what to produce 指导语言模型生成内容的文本

relative path 相对路径	Address from the current working directory 从当前工作目录出发的地址
SFTP SSH 文件传输协议	Secure file transfer running over SSH 运行在 SSH 之上的安全文件传输
set 集合	Unordered container of unique elements 元素唯一、无序的容器
short-circuiting 短路计算	Stopping evaluation once the result is decided 结果已定即停止求值
slice 切片	Sub-sequence selected by `seq[start:end:step]` 用 `seq[start:end:step]` 选出的子序列
SSH 安全外壳协议	Encrypted protocol for remote terminal access 用于远程终端访问的加密协议
thread 线程	Lightweight concurrent task sharing memory 共享内存的轻量并发任务
traceback 回溯	The printed roadmap of a crash 程序崩溃时打印的路线图
truthiness 真值	Whether a value behaves as True in a condition 值在条件判断中表现为真还是假
tuple 元组	Immutable sequence, written with commas 用逗号定义的不可变序列
with statement with 语句	Context manager guaranteeing cleanup 保证资源清理的上下文管理器

End of the Book

全书完

Now go and break something — and read the traceback.

去弄坏点什么吧——然后读懂那个 traceback。